

Prehľad UML Diagramov

SOFTVÉROVÁ ARCHITEKTÚRA



Cieľové skupiny

IKT architekti, analytici, vývojové tímy

1. Biznis analytici modelujúci procesy pomocou UML diagramov
2. Systémoví analytici navrhujúci štruktúru aplikácií a dátových modelov
3. Softvéroví architekti definujúci triedy, komponenty a ich vzťahy
4. Vývojári pracujúci s model-driven prístupmi (MDE, MDA)
5. Tímy používajúce UML nástroje (Enterprise Architect)
6. Architekti navrhujúci viacvrstvové a komponentové systémy
7. Špecialisti implementujúci transformácie modelov na kód
8. Tímy pracujúce s OCL pri validácii modelov
9. IT oddelenia zavádzajúce štandardy modelovania a dokumentácie
10. Organizácie formalizujúce návrh systémov pomocou UML metamodelu

Manažéri, konzultanti, rozhodovatelia

1. IT manažéri riadiaci kvalitu návrhu systémov a architektúr
2. Enterprise architekti nastavujúci modelovacie štandardy organizácie
3. Konzultanti hodnotiaci konzistenciu a správnosť modelov
4. CIO a CTO definujúci metodiky modelovania a dokumentácie
5. Projektoví manažéri koordinujúci návrh komplexných IT riešení
6. Organizácie implementujúce model-driven engineering
7. Firmy optimalizujúce vývoj pomocou modelovania a automatizácie
8. Spoločnosti vyžadujúce standardizovanú architektonickú dokumentáciu
9. Tímy pripravujúce sa na škálovanie systémov a architektúr
10. Rozhodovatelia požadujúci presné a validovateľné modely systémov



Čo sa naučíme?

1. Čo je UML diagram?
2. Na čo sa UML diagramy používajú v praxi?
3. Ktorým UML diagramom sa v praxi najčastejšie začína?
4. Aký je rozdiel medzi štruktúrnymi a behaviorálnymi UML diagramami?
5. Na čo slúži diagram tried?
6. Aký je rozdiel medzi diagramom tried a diagramom objektov?
7. Kedy použijeme sekvenčný diagram?
8. Na čo slúži diagram aktivít?
9. Aký je rozdiel medzi diagramom komponentov a diagramom nasadenia?
10. Prečo je UML užitočné aj vtedy, keď systém už existuje?

Čo je UML?

Štandardizovaný jazyk

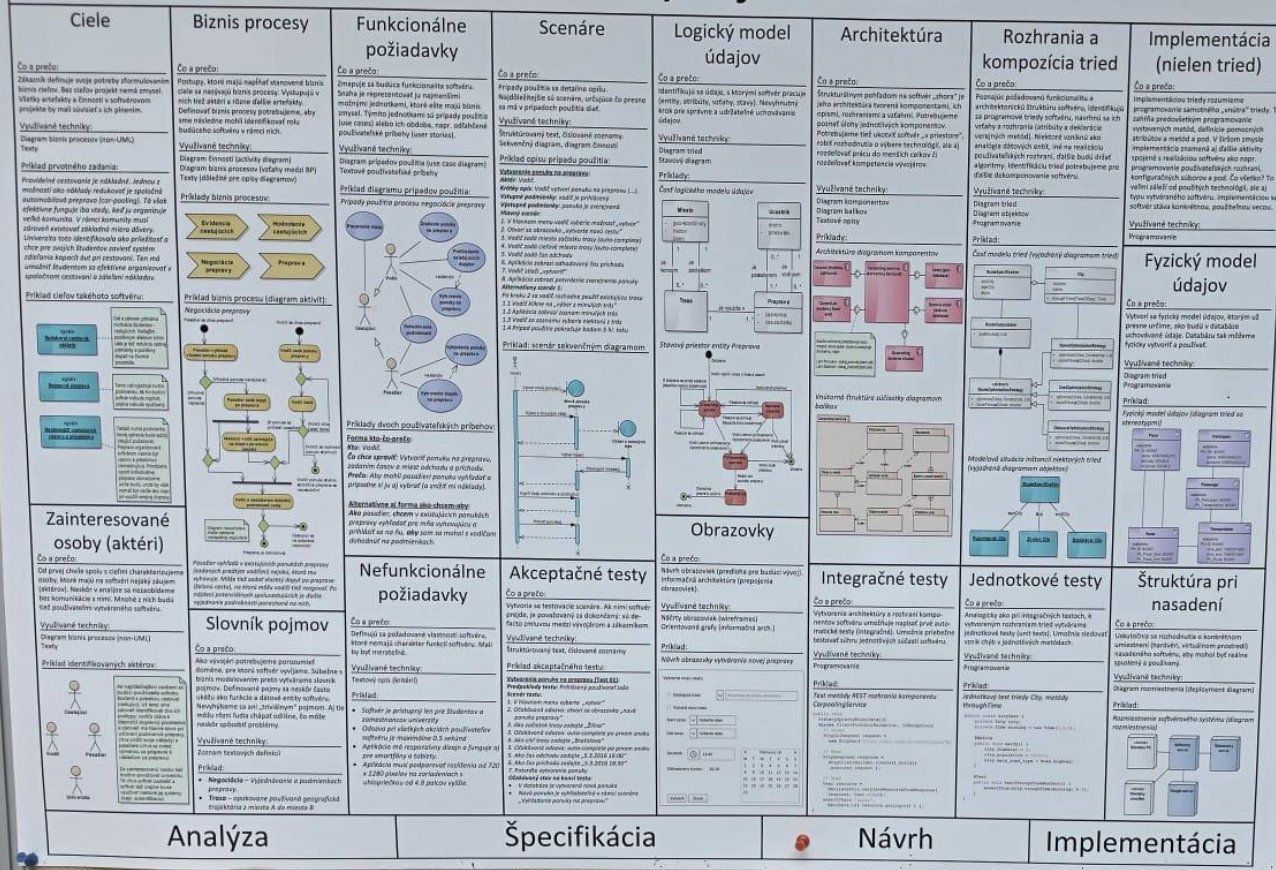
- UML (Unified Modeling Language) je vizuálny modelovací jazyk
- Definovaný organizáciou **OMG (Object Management Group)**
- Nejde len o sadu diagramov
- Je to formálne špecifikovaný jazyk s presne definovanou syntaxou a sémantikou

Prečo metamodel?

Bez metamodelu by UML bolo iba neformálnou kresliacou notáciou. Metamodel zabezpečuje:

1. Konzistenciu a jednoznačnosť modelov
2. Interpretovateľnosť nástrojmi
3. Automatizované spracovanie a validáciu
4. Možnosť transformácií modelov

Artefakty vo vývoji softvéru



Nová kancelária InQb na FEI STU !

Potrebuje odbornú pomoc pri založení firmy a rozbehnutí podnikania?

Na FEI STU bola zriadená pobočka Univerzitného technologického inkubátora InQb STU v rámci projektu Univerzitného inkubátora Science City STU.

Kontaktná kancelária sa nachádza v Bloku D v chodbe za výťahmi – miestnosť DP01

Individuálnu konzultáciu si môžete dohodnúť na kontaktoch:

info@inqb.sk, 0918 669 209

www.inqb.sk

Artefakty vo vývoji softvéru

Ciele

Čo a prečo:

Zákazník definuje svoje potreby sformulovaním biznis cieľov. Bez cieľov projekt nemá zmysel. Všetky artefakty a činnosti v softvérovom projekte by mali súvisieť s ich plnením.

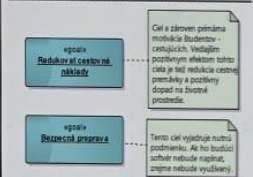
Využívané techniky:

Diagram biznis procesov (non-UML)
Texty

Príklad prvého zadania:

Pravidelné cestovanie je nákladné. Jednou z možností ako náklady redukovať je spoločná automobilová preprava (car-pooling). Tá však efektívne funguje iba vtedy, keď ju organizuje veľká komunita. V rámci komunity musí zároveň existovať základná miera dôvery. Univerzita toto identifikovala ako príležitosť a chce pre svojich študentov zaviesť systém zdieľania kapacít aut pri cestovaní. Ten má umožniť študentom sa efektívne organizovať v spoločnom cestovaní a zdieľaní nákladov.

Príklad cieľov takéhoto softvéru:



Zainteresované osoby (aktéri)

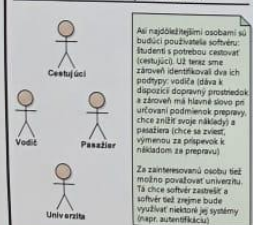
Čo a prečo:

Od prvej chvíľky spolu s cieľmi charakterizujeme osoby, ktoré majú na softvéri nejaký záujem (aktéri). Neskôr v analýze sa zaoberáme bez komunikácie s nimi. Mnohé z nich budú tiež používateľmi vytváraného softvéru.

Využívané techniky:

Diagram biznis procesov (non-UML)
Texty

Příklad identifikovaných aktériov:



Biznis procesy

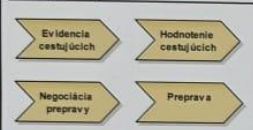
Čo a prečo:

Postupy, ktoré majú naplňovať stanovené biznis ciele sa nazývajú biznis procesy. Vystupujú v nich tiež aktéri a rôzne ďalšie artefakty. Definovať biznis procesy potrebujeme, aby sme následne mohli identifikovať rolu budúceho softvéru v rámci nich.

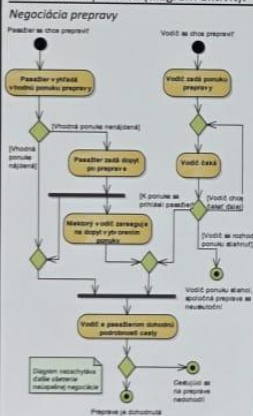
Využívané techniky:

Diagram činnosti (activity diagram)
Diagram biznis procesov (vzťahy medzi BP)
Texty (dôležité pre opisy diagramov)

Príklad biznis procesov:



Příklad biznis procesu (diagram aktivít):



Pasážier vyhľadá v existujúcich ponukách prepravy (zadaných predtým vodičmi) nejakú, ktorá mu vyhovuje. Má tiež zadať vlastný dopyt na prepravu (telo cestu), na ktorú má vodič tiež reagovať. Po nájdenej potenciálnej spolupráci je ďalšie vyjednanie podrobností ponechané na nich.

Slovník pojmov

Čo a prečo:

Ako vývojári potrebujeme porozumieť doméne, pre ktorú softvér vyvíjame. Súbežne s biznis modelovaním preto vytvárame slovník pojmov. Definované pojmy sa neskôr často ukážu ako funkcie a dátové entity softvéru. Nevýhoda sa ani „triviálnym“ pojmom. Aj tie môžu rôzni ľudia chápať odlišne, čo môže viesť k problémom.

Využívané techniky:

Zoznam textových definícií

- Příklad:**
- Negociácia – Vyjednanie o podmienkach prepravy.
 - Trasa – opakovane používaná geografická trajektória z miesta A do miesta B.

Funkcionálne požiadavky

Čo a prečo:

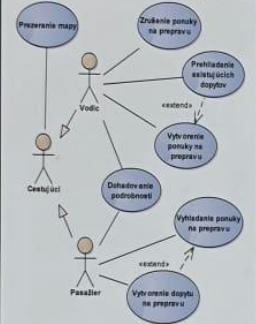
Zmapuje sa budúca funkcionálna softvéru. Snaha je reprezentovať ju najmenšími možnými jednotkami, ktoré ešte majú biznis zmysel. Týmto jednotkami sú prípady použitia (use cases) alebo ich obdoba, napr. odľahčené používateľské príbehy (user stories).

Využívané techniky:

Diagram prípadov použitia (use case diagram)
Textové používateľské príbehy

Příklad diagramu prípadov použitia:

Prípady použitia procesu negociácie prepravy



Příklad dvoch používateľských príbehov:

Forma kto-čo-prečo:
Kto: Vodič.
Čo chce spraviť: Vytvoriť ponuku na prepravu, zadaním času a miest odchodu a príchodu.
Prečo: Aby mohli pasažieri ponuku vyhľadať a prípadne si ju aj vybrať (a znížiť náklady).

Alternatívne aj forma ako-keď-aby:
Ako pasažier, **keď** v existujúcich ponukách prepravy vyhľadám pre mňa vyhovujúcu a **príhlásiť sa na ňu, aby som sa mohol s vodičom dohodnúť na podmienkach.**

Nefunkcionálne požiadavky

Čo a prečo:

Definujú sa požadované vlastnosti softvéru, ktoré nemajú charakter funkcií softvéru. Mali by byť merateľné.

Využívané techniky:

Textový opis (kritériá)

Příklad:

- Softvér je prístupný len pre študentov a zamestnancov univerzity.
- Odpoveď na všetky akcie používateľov softvéru je maximálne 0.5 sekundy.
- Aplikácia má responzívny dizajn a funguje aj pre smartfóny a tablety.
- Aplikácia musí podporovať rozlíšenia od 720 x 1280 pixelov na zariadeniach s uhlopriečkou od 4.8 palcov vyššie.

Scenáre

Čo a prečo:

Prípady použitia sa detailne opíšu. Najdôležitejšie sú scenáre, určujúce čo presne sa má v prípadoch použitia diať.

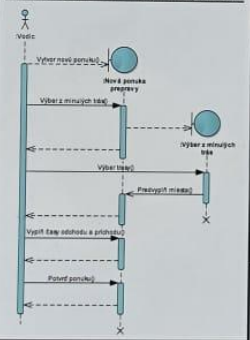
Využívané techniky:

Štruktúrovaný text, číslované zoznamy.
Sekvenčný diagram, diagram činnosti

Příklad opisu prípadu použitia:

Vytvorenie ponuky na prepravu:
Akteři: Vodič.
Kritériá opisu: Vodič vytvorí ponuku na prepravu (...).
Vstupné podmienky: vodič je prihlásený.
Výstupné podmienky: ponuka je zverejnená.
Hlavný scenár:
1. V hlavnom menu vodič vyberie možnosť „vytvor“
2. Otvorí sa obrazovka „vytvor novú cestu“
3. Vodič zadá miesto začiatku trasy (auto-complete)
4. Vodič zadá cieľové miesto trasy (auto-complete)
5. Vodič zadá čas odchodu
6. Aplikácia zobrazí odhadovaný čas príchodu
7. Vodič stlačí „vytvor“
8. Aplikácia zobrazí potvrdenie zverejnenia ponuky.
Alternatívny scenár 1:
Po kroku 2 sa vodič rozhodne použiť existujúcu trasu.
1.1 Vodič klikne na „vyber z minulých trás“
1.2 Aplikácia zobrazí zoznam minulých trás
1.3 Vodič po zozname vyberie niektorú z trás
1.4 Prípady použitia pokračujú bodom 5 hl. toku

Příklad: scenár sekvenčným diagramom



Akceptačné testy

Čo a prečo:

Vytvoria sa testovacie scenáre. Ak nimi softvér prejde, je považovaný za dokončený: sú de-facto zmluvou medzi vývojárom a zákazníkom.

Využívané techniky:

Štruktúrovaný text, číslované zoznamy

Příklad akceptačného testu:

Vytvorenie ponuky na prepravu [Test 01]:
Pré-kondície: Test: Prihlásený používateľ / do scenára testu:
1. V hlavnom menu vyberiete „vytvor“
2. Očakávaná odpoveď: otvorí sa obrazovka „nová ponuka prepravy“
3. Ako začiatok trasy zadáte „Žilina“
4. Očakávaná odpoveď: auto-complete po prvom znaku
4. Ako cieľ trasy zadáte „Bratislava“
5. Očakávaná odpoveď: auto-complete po prvom znaku
5. Ako čas odchodu zadáte „5.3.2016 16:00“
6. Ako čas príchodu zadáte „5.3.2016 18:30“
7. Potvrďte vytvorenie ponuky
Očakávaný stav na konci testu:
• V databáze je vytvorená nová ponuka
• Nová ponuka je vyhľadateľná v rámci scenára „Vyhľadanie ponuky na prepravu“

Logický model údajov

Čo a prečo:

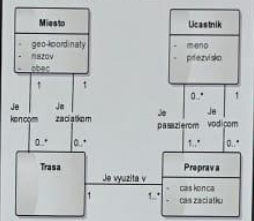
Identifikujú sa údaje, s ktorými softvér pracuje (entity, atribúty, vzťahy, stavy). Nevýhoda: krok pre správne a udržateľné uchovávanie údajov.

Využívané techniky:

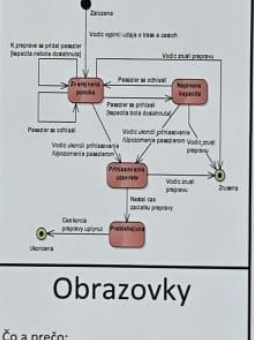
Diagram tried
Stavový diagram

Příklad:

Časť logického modelu údajov



Stavový priestor entity Preprava



Obrazovky

Čo a prečo:

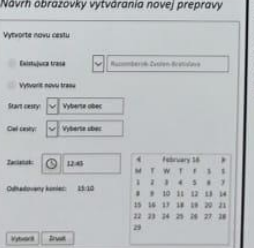
Návrh obrazoviek (predloha pre budúci vývoj). Informačná architektúra (porepania obrazoviek).

Využívané techniky:

Náčrty obrazoviek (wireframes)
Orientované grafy (informačná arch.)

Příklad:

Návrh obrazovky vytvárania novej prepravy



Architektúra

Čo a prečo:

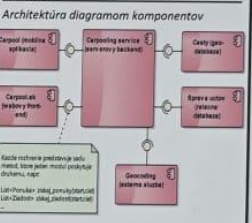
Štruktúrálnym pohľadom na softvér „zhora“ je jeho architektúra tvorená komponentami, ich opismi, rozhraniami a vzťahmi. Potrebujeme poznať úlohy jednotlivých komponentov. Potrebujeme tiež ukotviť softvér „v priestore“, robí rozhodnutia o výbere technológií, ale aj rozdeľovať prácu do menších celkov či rozdeľovať kompetencie vývojárov.

Využívané techniky:

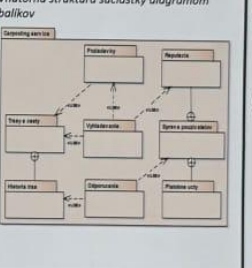
Diagram komponentov
Diagram balíkov
Textové opisy

Příklad:

Architektúra diagramu komponentov



Vnútroštruktúra súčastí diagramom balíkov



Integračné testy

Čo a prečo:

Vytvorenie architektúry a rozhraní komponentov softvéru umožňuje napísať prvé automatické testy (integračné). Umožnia priebežne testovať súhrn jednotlivých súčastí softvéru.

Využívané techniky:

Programovanie

Příklad:

Test metódy REST rozhrania komponentu CarpoolingService

```
public void testCreateTransport() {
    // ...
    // ...
    // ...
}
```

Rozhrania a kompozícia tried

Čo a prečo:

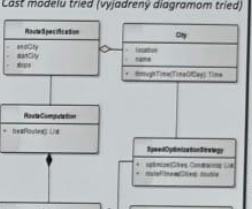
Poznajúc požadovanú funkcionálnu a architektonickú štruktúru softvéru, identifikujú sa programové triedy softvéru, navrhnu sa ich vzťahy a rozhrania (atribúty a deklarácie verejných metód). Niektoré vzniknú ako analógie dátových entít, iné na realizáciu používateľských rozhraní, ďalšie budú držať algoritmy. Identifikáciu tried potrebujeme pre ďalšie dekomponovanie softvéru.

Využívané techniky:

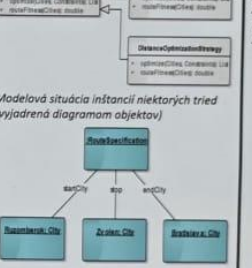
Diagram tried
Diagram objektov
Programovanie

Příklad:

Časť modelu tried (vyjadrený diagramom tried)



Modelová situácia inštancií niektorých tried (vyjadrená diagramom objektov)



Jednotkové testy

Čo a prečo:

Analogicky ako pri integračných testoch, k vytvoreniu rozhraníam tried vytvárame jednotkové testy (unit tests). Umožnia priebežne testovať súhrn jednotlivých súčastí softvéru.

Využívané techniky:

Programovanie

Příklad:

Jednotkový test triedy City, metódy throughTime

```
public void testThroughTime() {
    // ...
    // ...
    // ...
}
```

Implementácia (nielen tried)

Čo a prečo:

Implementáciou triedy rozumieme programovanie samotného „vnútra“ triedy. To zahŕňa predovšetkým programovanie vystavených metód, definície pomocných atribútov a metód a pod. V širšom zmysle implementácia znamená aj ďalšie aktivity spojené s realizáciou softvéru ako napr. programovanie používateľských rozhraní, konfiguračných súborov a pod. Čo všetko? To veľmi záleží od použitých technológií, ale aj typu vytváraného softvéru. Implementáciou sa softvér stáva konkrétnou, použiteľnou vecou.

Využívané techniky:

Programovanie

Fyzický model údajov

Čo a prečo:

Vytvorí sa fyzický model údajov, ktorým už presne určíme, ako budú v databáze uchovávané údaje. Databázu tak môžeme fyzicky vytvoriť a používať.

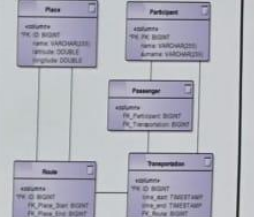
Využívané techniky:

Diagram tried

Programovanie

Příklad:

Fyzický model údajov (diagram tried so stereotypmi)



Štruktúra pri nasadení

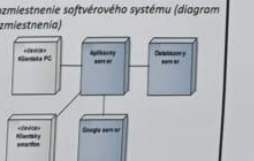
Čo a prečo:

Uskutočnia sa rozhodnutia o konkrétnom umiestnení (hardvéri, virtuálnom prostredí) nasadeného softvéru, aby mohol byť reálne spustený a používaný.

Využívané techniky:

Diagram rozmiestnenia (deployment diagram)

Příklad:



Analýza

Špecifikácia

Návrh

Implementácia

ABOUT THE UNIFIED MODELING LANGUAGE SPECIFICATION VERSION 2.5.1

2.5.1 • UML • SPECIFICATIONS

UML® — Unified Modeling Language

A specification defining a graphical language for visualizing, specifying, constructing, and documenting the artifacts of distributed object systems.



Title: Unified Modeling Language

Acronym: UML®

Version: 2.5.1

Document Status: formal ⓘ

Publication Date: December 2017

Categories: [Modeling](#) [Software Engineering](#) [Platform](#)

IPR Mode ⓘ RF-Limited ⓘ

TABLE OF CONTENTS

- [About the Specification](#)
- [Issues associated with this specification](#)
- [Specification Documents](#)
 - [Normative Documents](#)
 - [Normative Machine Readable Documents](#)
 - [Informative Documents](#)
- [History](#)
 - [Formal Versions](#)

ISSUES ASSOCIATED WITH THIS SPECIFICATION



Issues Reported in this Specification - *Version 2.5.1 only*



Issues Fixed in this Specification - *Version 2.5.1 only*



Report an issue



Issue Tracker (Members Only)

SPECIFICATION DOCUMENTS

NORMATIVE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
Specification	PDF	UML/2.5.1/PDF	formal/17-12-05

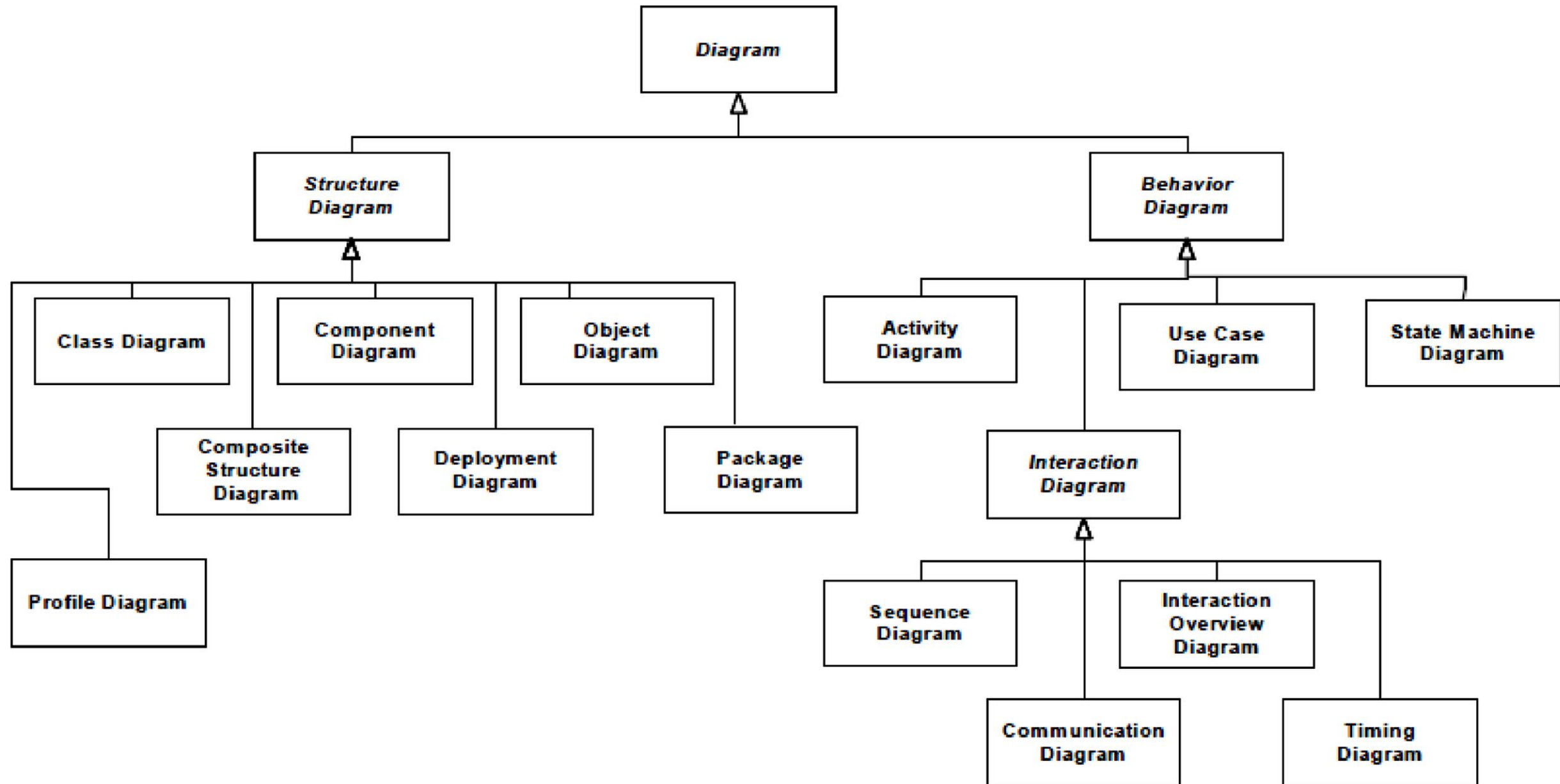
NORMATIVE MACHINE READABLE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
UML 2.5.1 Abstract Syntax Metamodel	XMI	UML/20161101/UML.xmi	ptc/18-01-01
UML 2.5.1 Primitive Types	XMI	UML/20161101/PrimitiveTypes.xmi	ptc/18-01-02
UML 2.5.1 Standard Profile	XMI	UML/20161101/StandardProfile.xmi	ptc/18-01-03
UML 2.5.1 Diagram Interchange Metamodel	XMI	UML/20161101/UMLDI.xmi	ptc/18-01-04

INFORMATIVE DOCUMENTS



Taxonómia štruktúrálnych a behaviorálnych diagramov



File

Home

Convert

Edit

Fill & Sign

Organize

Comment

View

Form

Protect

Share

Connect

Accessibility

Help

Tell me what you want to do...

Start

Unified Modeling Langu...

1 Scope

2 Conformance

3 Normative References

4 Terms and Definitions

5 Notational Conventions

6 Additional Information

7 Common Structure

8 Values

9 Classification

10 Simple Classifiers

11 Structured Classifiers

12 Packages

13 Common Behavior

14 StateMachines

15 Activities

16 Actions

17 Interactions

18 UseCases

19 Deployments

20 InformationFlows

21 Primitive Types

22 Standard Profile

Annex A: Diagrams

Annex B: UML Diagram Interchange

Annex C: Keywords

Annex D: Tabular Notation for Sequenc

Annex E: XMI Serialization and Schema

Unified Modeling Language, v2.5.1 - Foxit PhantomPDF

Find

As is shown in Figure A.5, there are two major kinds of diagram types: structure diagrams and behavior diagrams.

```
graph BT; Diagram --> StructureDiagram[Structure Diagram]; Diagram --> BehaviorDiagram[Behavior Diagram]; StructureDiagram --> ClassDiagram[Class Diagram]; StructureDiagram --> ComponentDiagram[Component Diagram]; StructureDiagram --> ObjectDiagram[Object Diagram]; StructureDiagram --> PackageDiagram[Package Diagram]; ComponentDiagram --> CompositeStructureDiagram[Composite Structure Diagram]; ComponentDiagram --> DeploymentDiagram[Deployment Diagram]; ClassDiagram --> ProfileDiagram[Profile Diagram]; BehaviorDiagram --> ActivityDiagram[Activity Diagram]; BehaviorDiagram --> UseCaseDiagram[Use Case Diagram]; BehaviorDiagram --> StateMachineDiagram[State Machine Diagram]; ActivityDiagram --> InteractionDiagram[Interaction Diagram]; InteractionDiagram --> SequenceDiagram[Sequence Diagram]; InteractionDiagram --> InteractionOverviewDiagram[Interaction Overview Diagram]; InteractionDiagram --> CommunicationDiagram[Communication Diagram]; InteractionDiagram --> TimingDiagram[Timing Diagram];
```

Figure A.5 The taxonomy of structure and behavior diagrams

Structure diagrams show the static structure of the objects in a system. That is, they depict those elements in a specification that are irrespective of time. The elements in a structure diagram represent the meaningful concepts of an application, and may include abstract, real-world and implementation concepts. For example, a structure diagram for an airline reservation system might include classifiers that represent seat assignment algorithms, tickets, and a credit authorization service. Structure diagrams do not show the details of dynamic behavior, which are illustrated by behavioral diagrams. However, they may show relationships to the behaviors of the classifiers exhibited in the structure diagrams.

Behavior diagrams show the dynamic behavior of the objects in a system, including their methods, collaborations, activities, and state histories. The dynamic behavior of a system can be described as a series of changes to the system over time. Behavior diagrams can be further classified into several other kinds as illustrated in Figure A.5.

NOTE.

This taxonomy provides a logical organization for the various major kinds of diagrams. However, it does not preclude mixing different kinds of diagram types, as one might do when one combines structural and behavioral elements (e.g., showing a state machine nested inside an internal structure). Consequently, the boundaries between the various kinds of diagram types are not strictly enforced.

The constructs contained in each of the UML diagrams are described in the clauses indicated below.

- Activity Diagram - Activities
- Class Diagram - Structured Classifiers
- Communication Diagram - Interactions
- Component Diagram - Structured Classifiers
- Composite Structure Diagram - Structured Classifiers
- Deployment diagram - Deployments
- Interaction Overview Diagram - Interactions
- Object Diagram - Classification
- Package Diagram - Packages

727 / 796

150%

UML a 14 typov diagramov



Štrukturálne diagramy (Structure Diagrams) – 7 typov

1. Diagram tried (Class Diagram)
2. Diagram objektov (Object Diagram)
3. Diagram komponentov (Component Diagram)
4. Diagram nasadenia (Deployment Diagram)
5. Diagram balíkov (Package Diagram)
6. Diagram kompozitnej štruktúry (Composite Structure Diagram)
7. Diagram profilov (Profile Diagram)



Behaviorálne diagramy (Behavior Diagrams) – 7 typov

- (a) Základné behaviorálne diagramy:
8. Diagram prípadov použitia (Use Case Diagram)
 9. Diagram aktivít (Activity Diagram)
 10. Stavový diagram (State Machine Diagram)
- (b) Interakčné diagramy (Interaction Diagrams):
11. Sekvenčný diagram (Sequence Diagram)
 12. Komunikačný diagram (Communication Diagram)
 13. Diagram prehľadu interakcií (Interaction Overview Diagram)
 14. Diagram časovania (Timing Diagram)

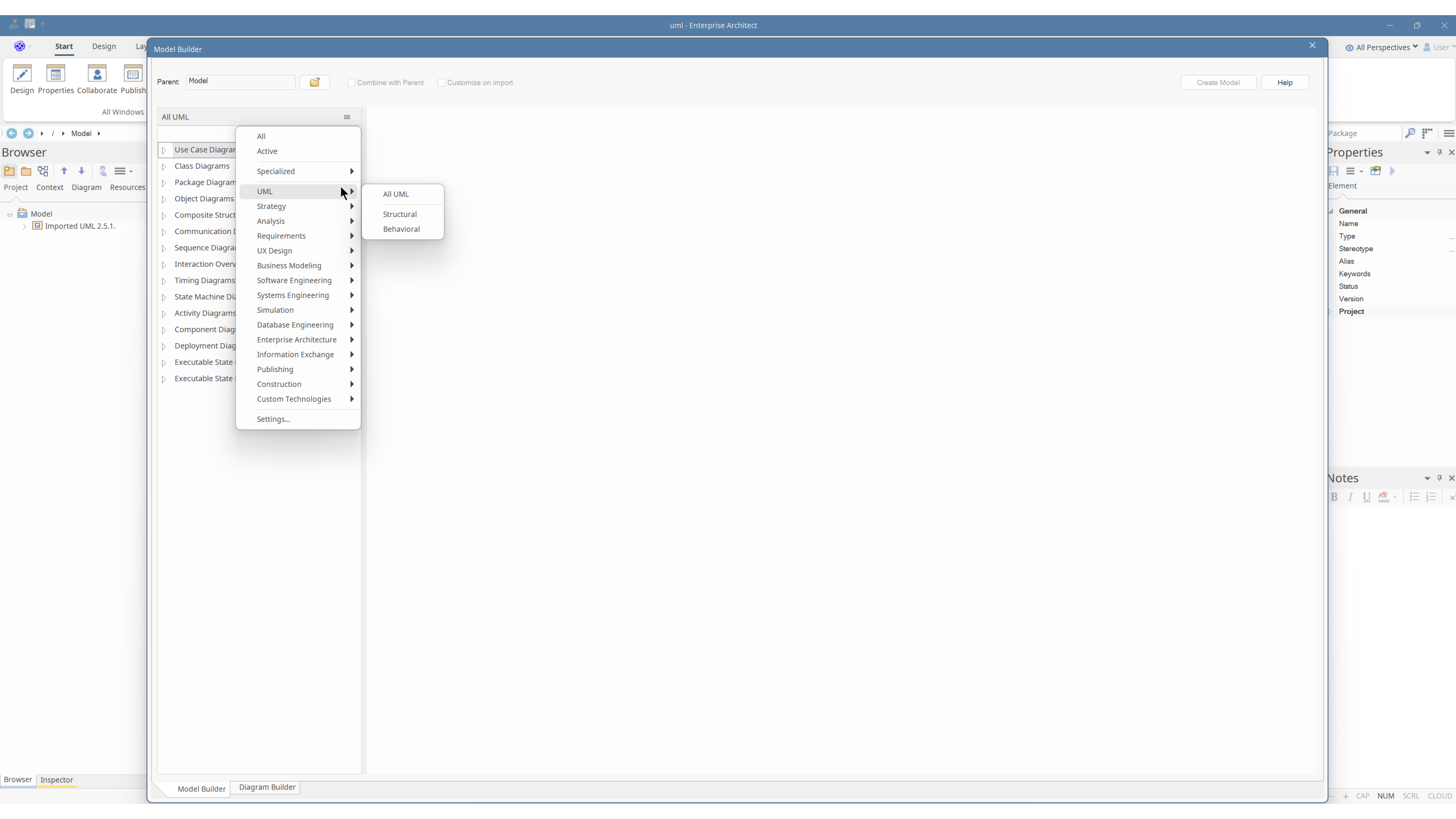


Diagram tried (Class Diagram)

Zobrazuje triedy systému, ich atribúty, metódy a vzťahy. Patrí medzi **najčastejšie používané diagramy** pri návrhu softvéru.

Pre koho

Analytici,
architekti,
programátori,
testerí,
študenti OOP

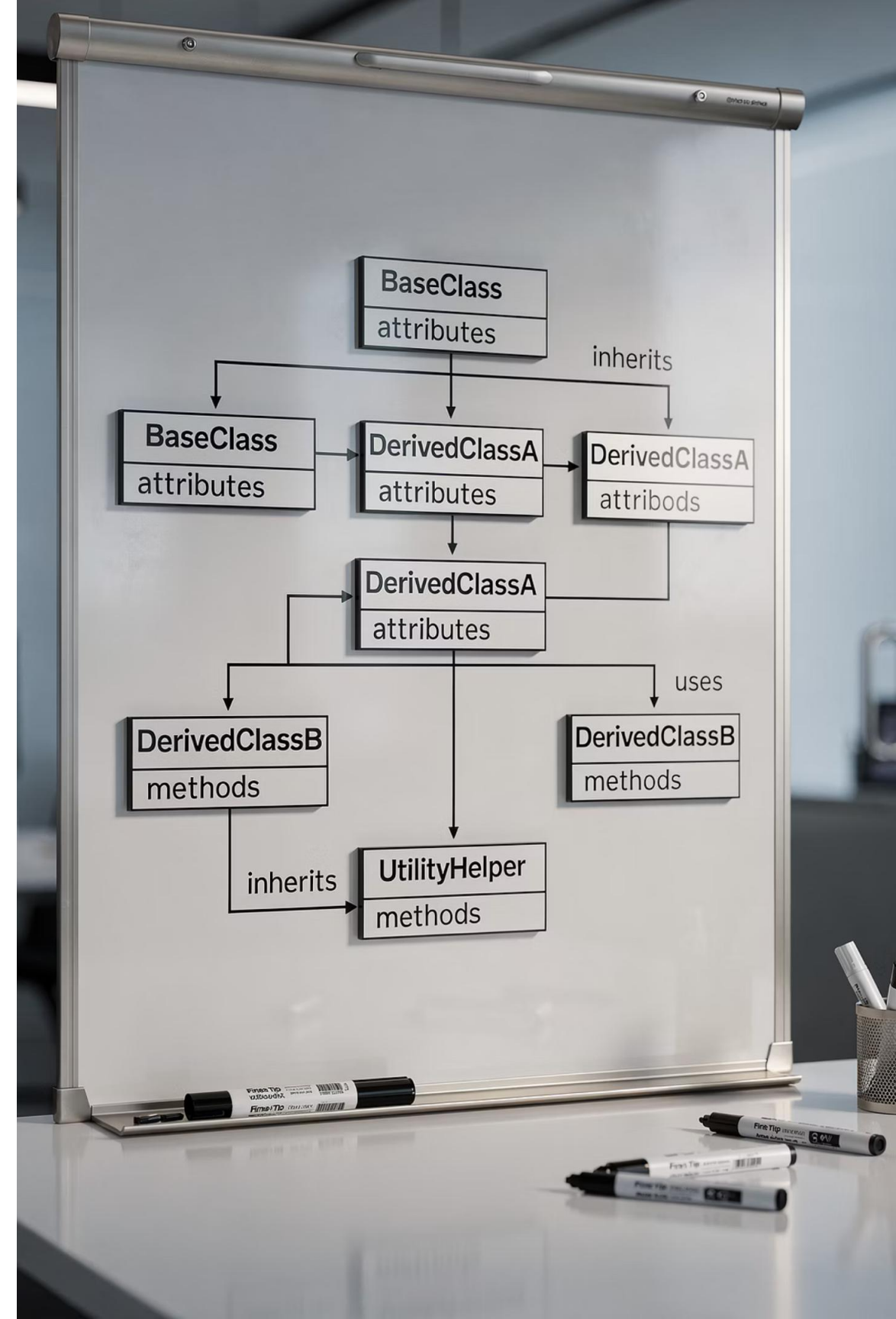
Kde a kedy

Návrh IS,
webových a
mobilných
aplikácií; vo fáze
analýzy a návrhu

Prečo

Jasné pochopenie
štruktúry, zníženie
rizika chýb, lepšia
komunikácia tímu

V diagrame definujeme triedy, vlastnosti, operácie a väzby ako asociáciu, dedičnosť, agregáciu alebo kompozíciu, ktoré slúžia ako podklad pre implementáciu.



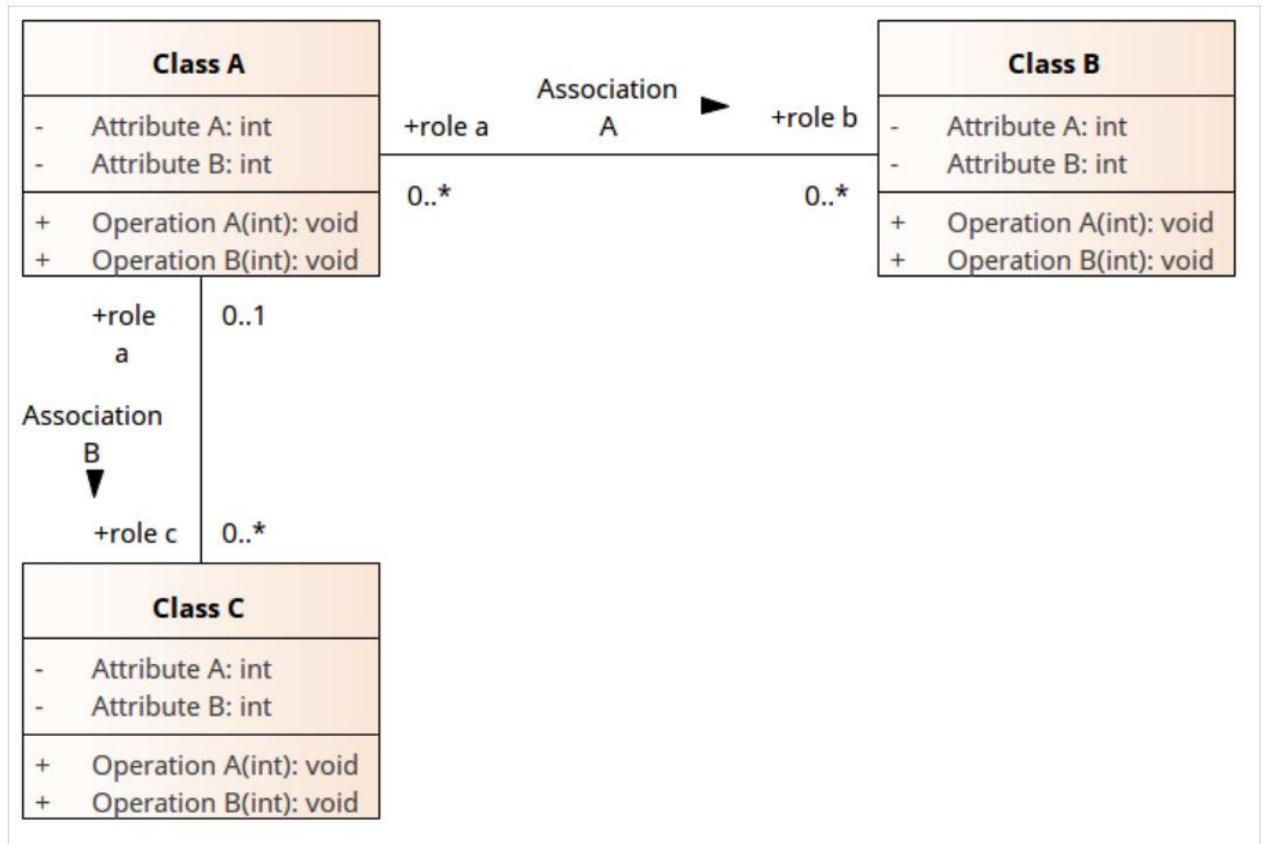
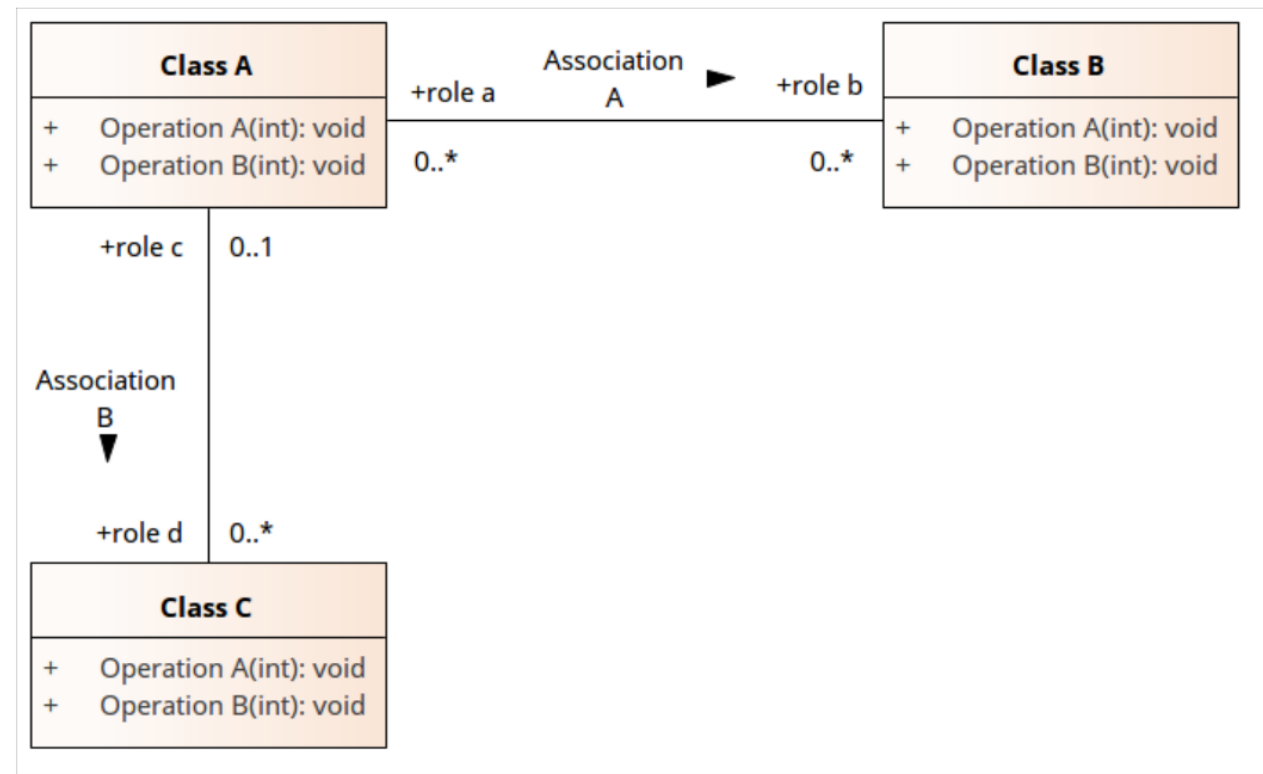
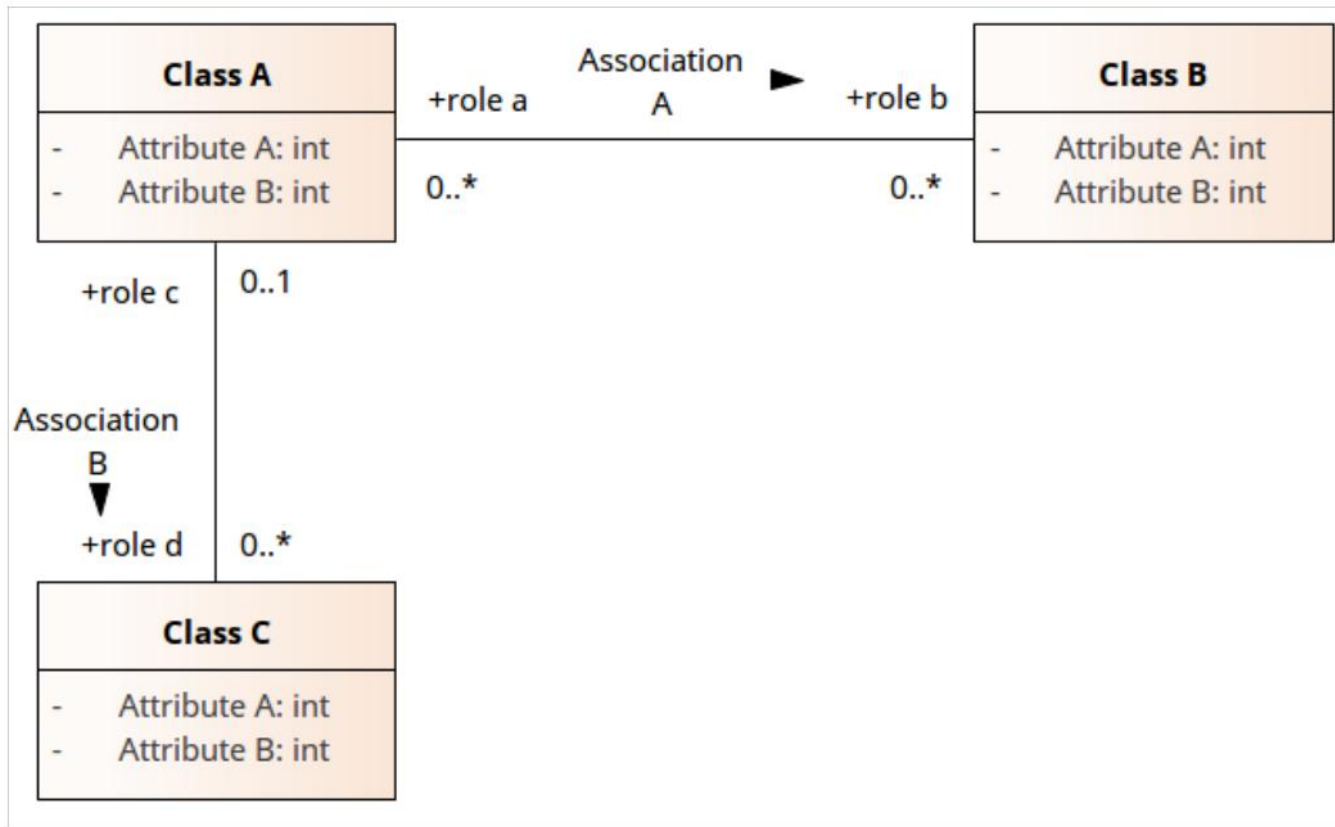


Diagram objektov (Object Diagram)

Zobrazuje **konkrétne objekty a ich vzťahy** v určitom okamihu behu systému ako praktická ukážka inštancií tried.

Objekt je pomenovaný ako
názovObjektu : názovTriedy.

Používa sa pri vysvetľovaní konkrétnych stavov systému, pri testovaní, ladení a overovaní správnosti objektového modelu.

Pre koho

Analytici, vývojári, tester, študenti

Kedy

Keď nestačí všeobecný diagram tried a potrebujeme konkrétny príklad

Prečo

Overenie, či triedny model funguje na príkladoch z praxe

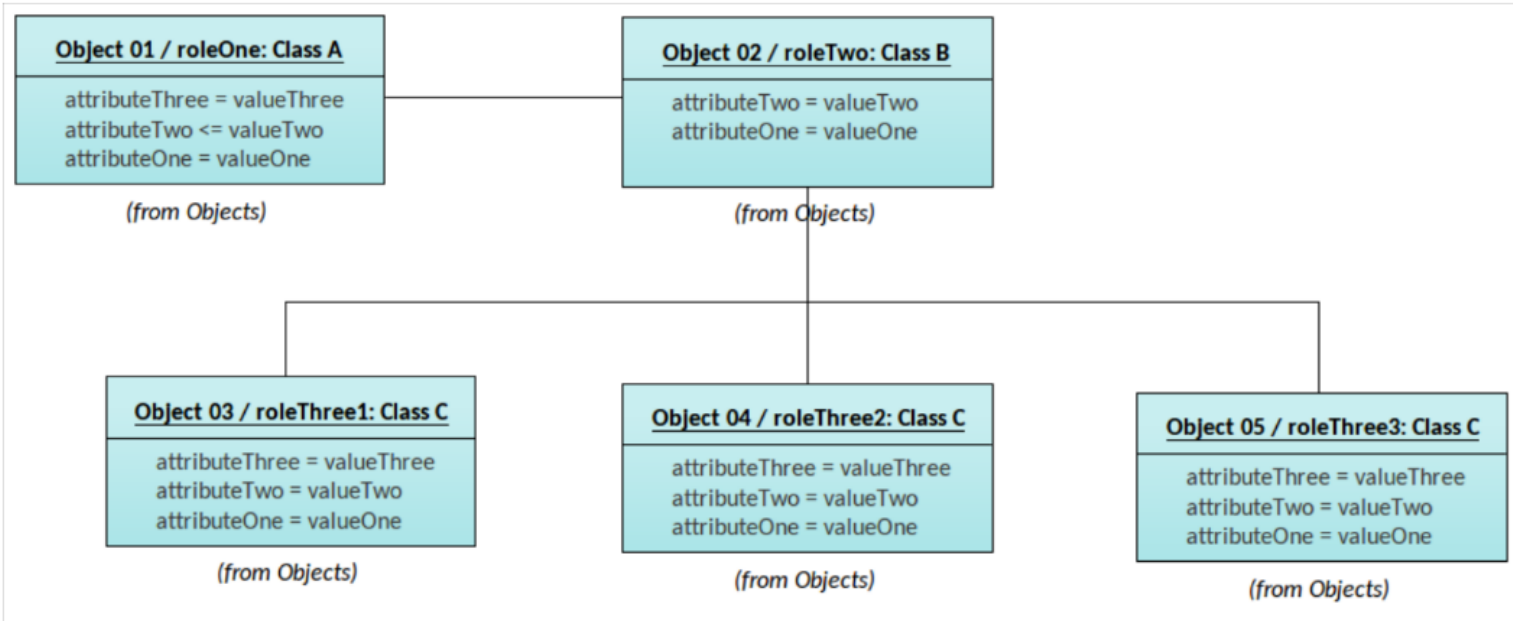
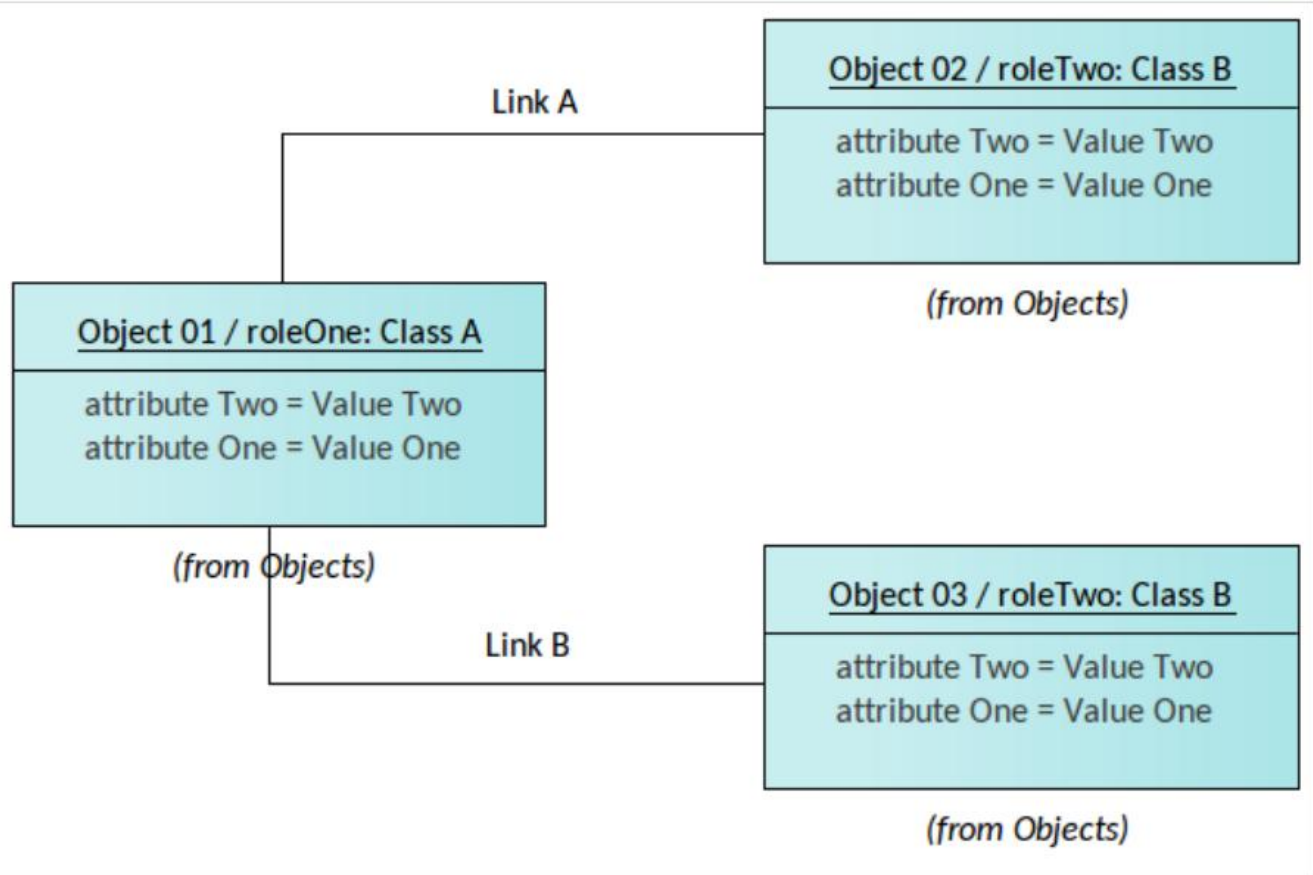


Diagram komponentov (Component Diagram)

Zobrazuje **softvérové komponenty systému** a ich **vzájomné závislosti**. Komponent môže predstavovať modul, knižnicu, službu alebo časť systému.



Pre koho

Softvéroví architekti, vývojári, DevOps špecialisti, systémoví analytici, projektoví manažéri



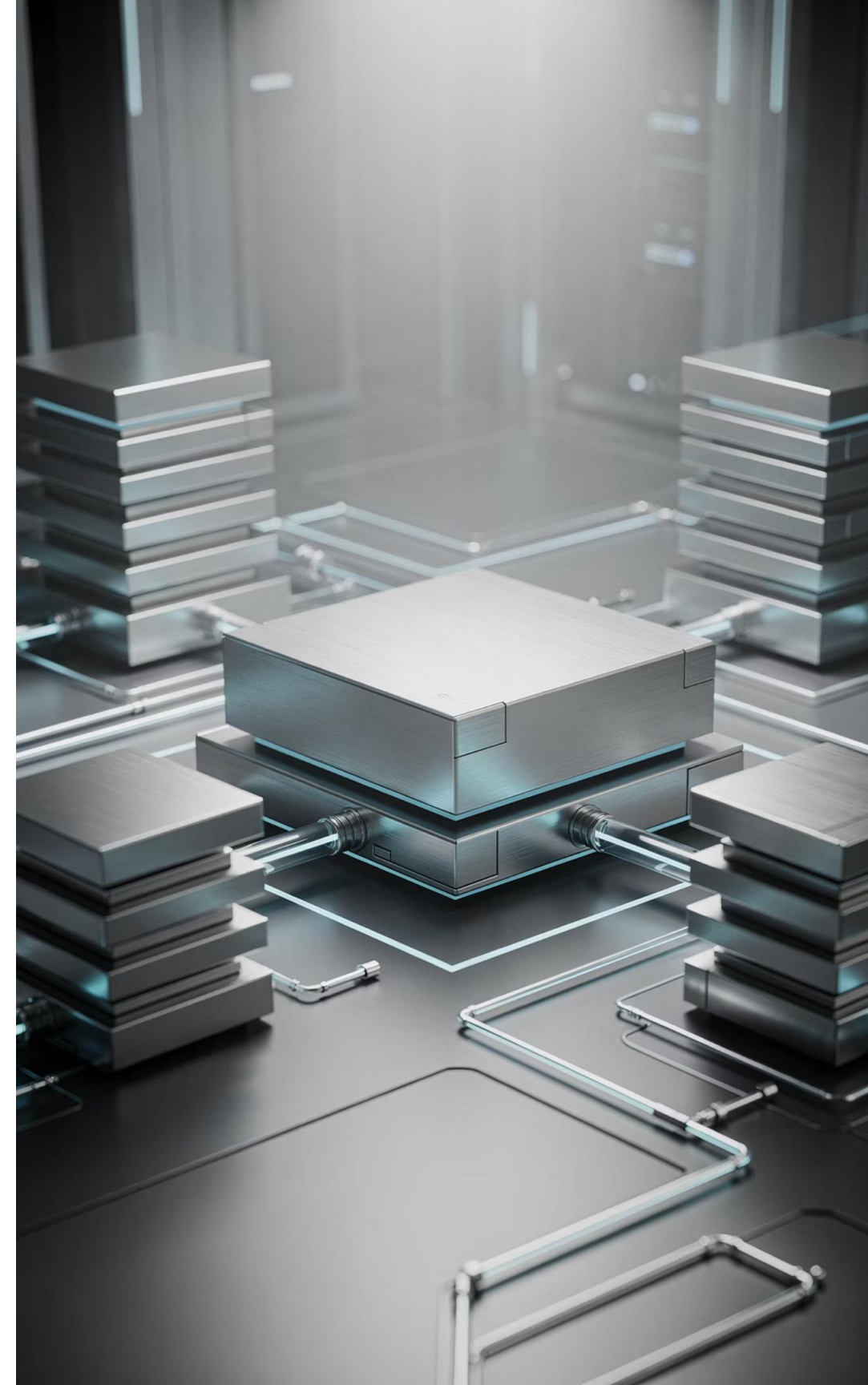
Kde a kedy

Návrh architektúry, mikroslužby, refaktoring väčších systémov; vo fáze architektonického návrhu



Prečo

Rozdelenie systému na zrozumiteľné časti, lepšie riadenie závislostí, uľahčenie údržby a nasadenia



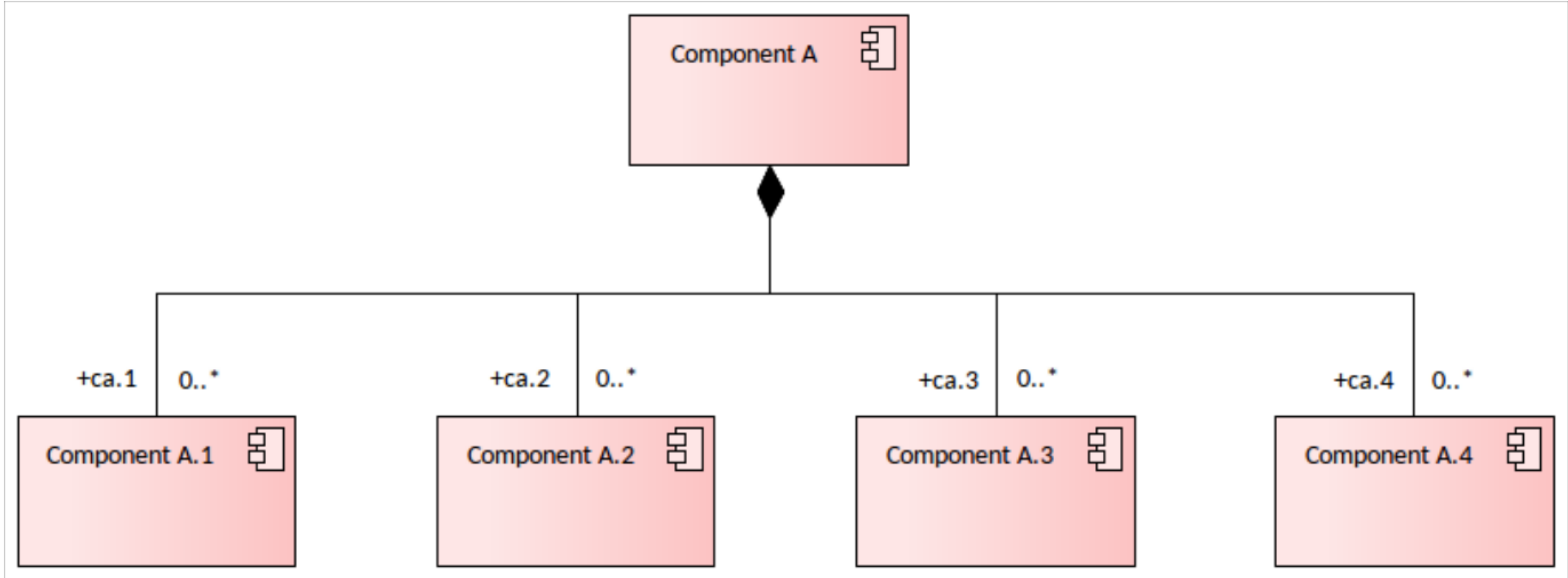
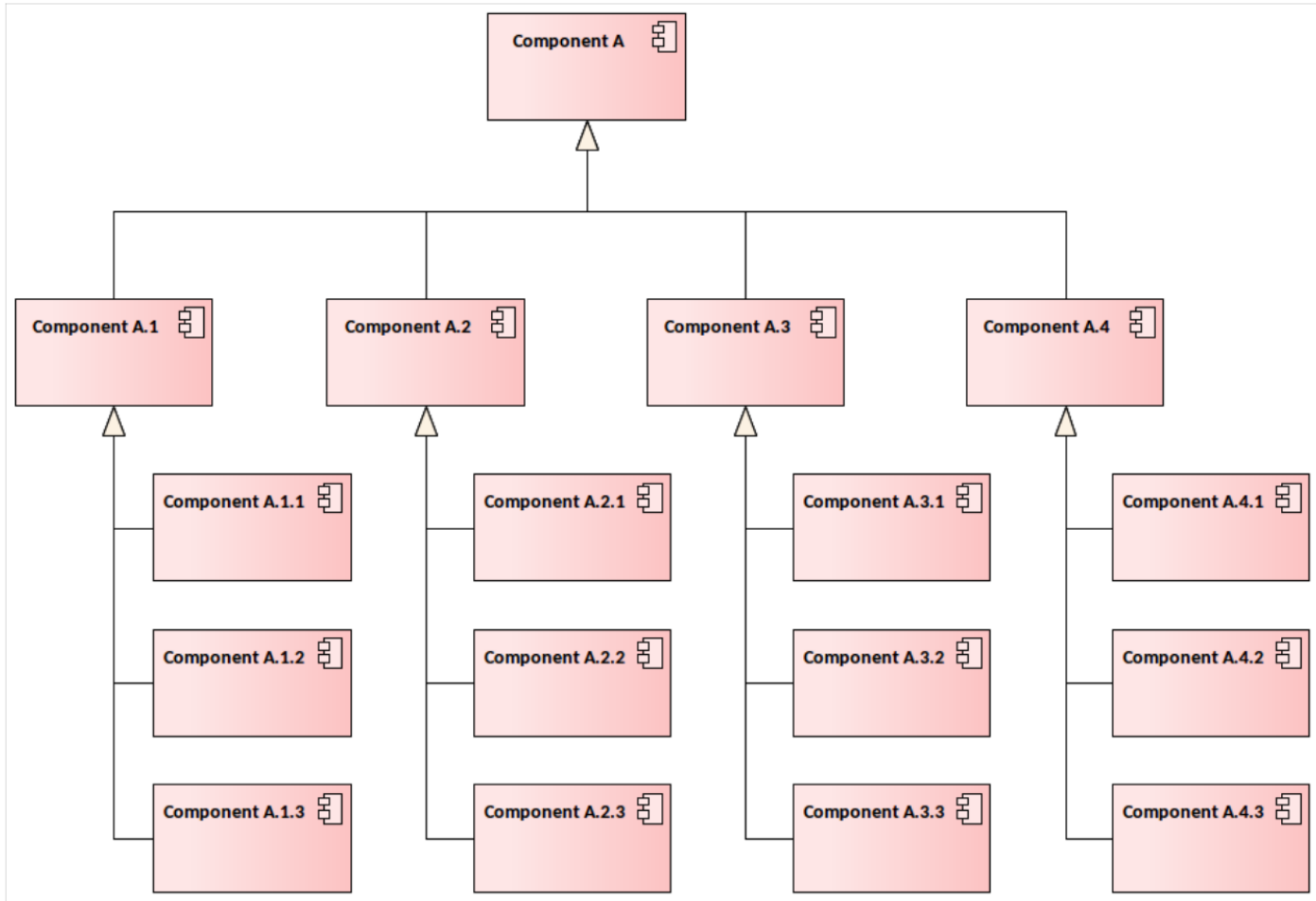
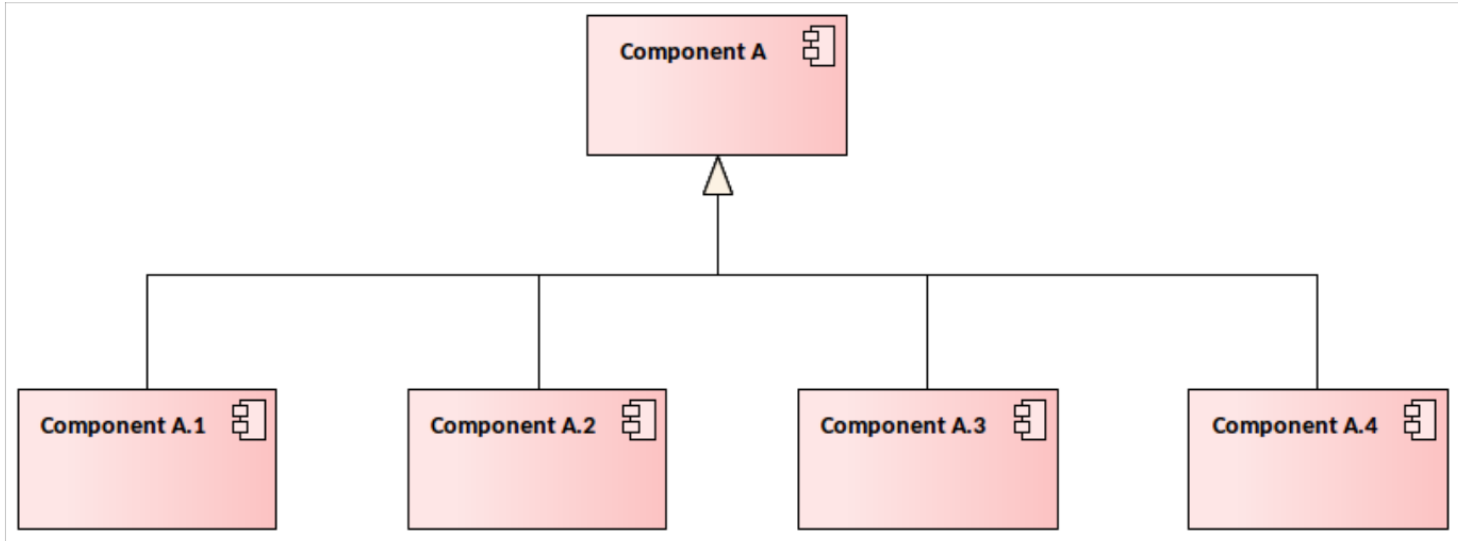


Diagram nasadenia (Deployment Diagram)



Zobrazuje **fyzické alebo virtuálne prostredie**, v ktorom bude **softvérový systém spustený** ako servery, zariadenia, uzly, databázy a ich prepojenia.

Kde sa používa

Infraštruktúra, cloud, distribuované systémy, klient-server architektúra

Kedy sa používa

Pred nasadením, pri migrácii do cloudu, pri dokumentovaní produkčného prostredia

Prečo sa používa

Správny návrh infraštruktúry, výkon, dostupnosť, bezpečnosť a komunikácia medzi časťami

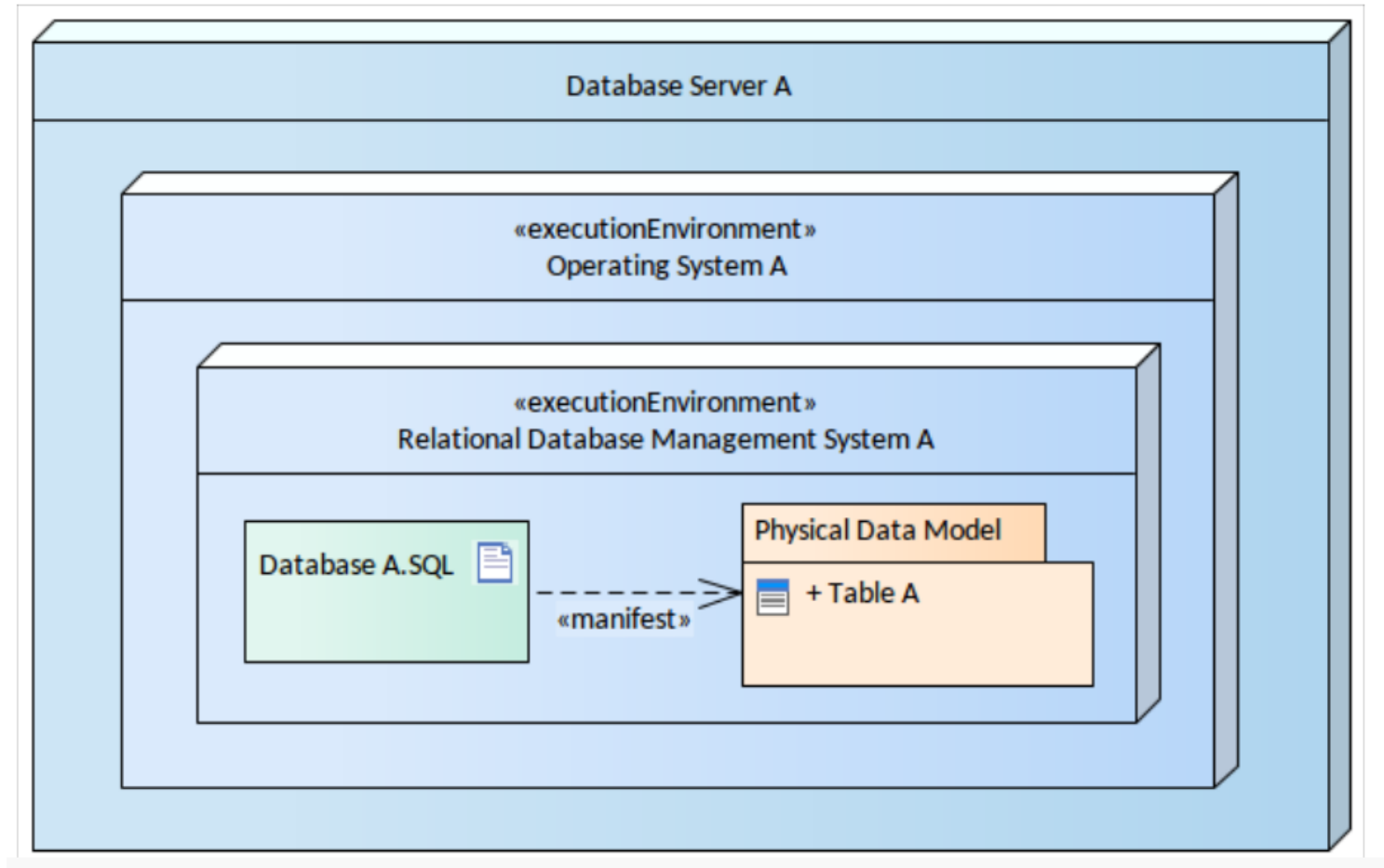
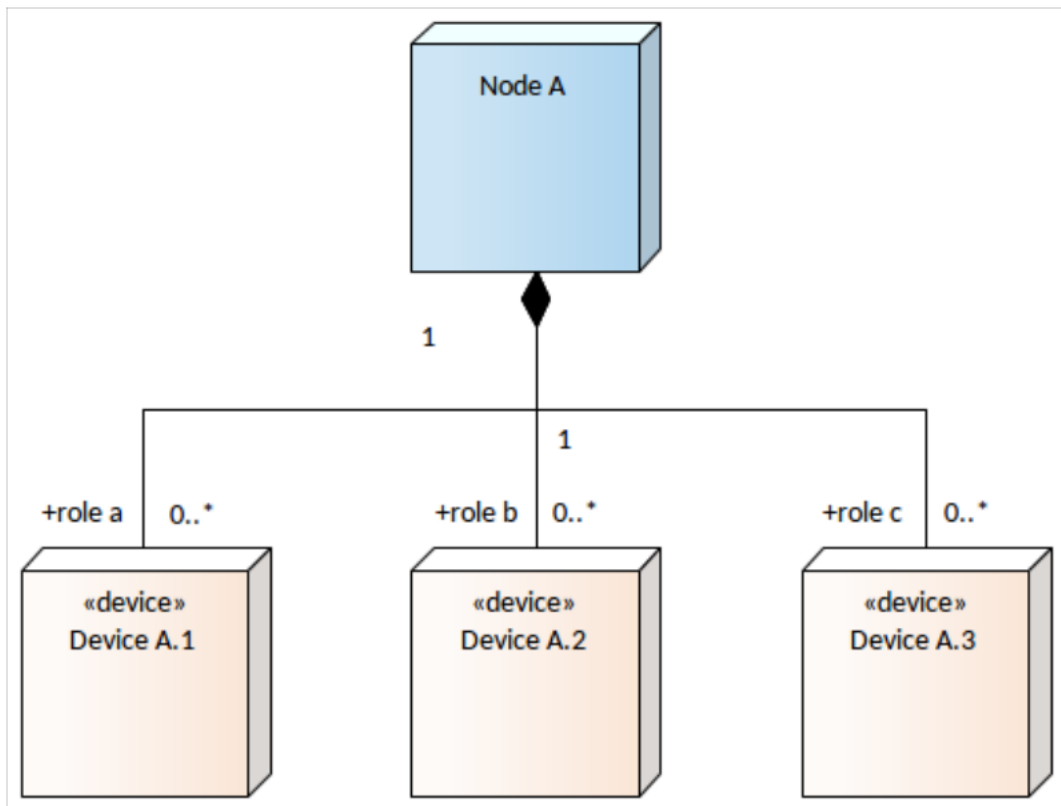
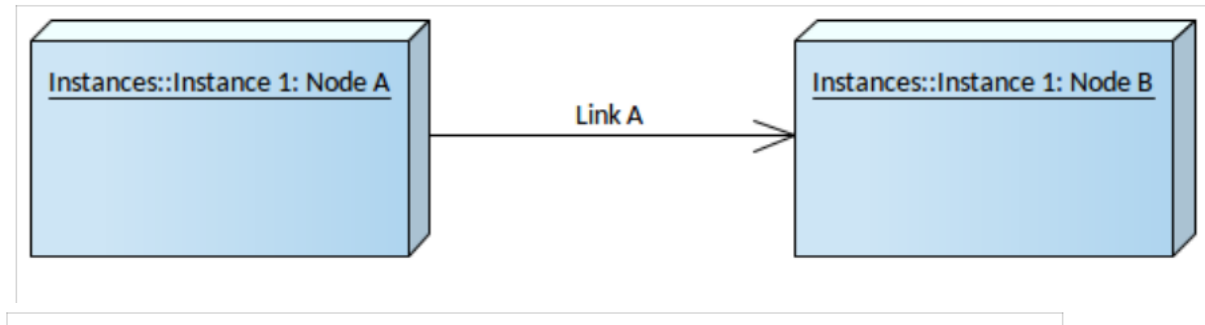


Diagram balíkov (Package Diagram)

Zobrazuje **organizáciu modelu** alebo **systemu** do **logických skupín – balíkov**, ktoré môžu obsahovať triedy, rozhrania, komponenty alebo iné UML prvky.

Pre koho

Architekti, analytici, vývojári a tímy pracujúce s rozsiahlejšími systémami

Kde a kedy

Organizácia veľkých projektov, rozdelenie na vrstvy a moduly; keď je systém rozsiahly a neprehľadný

Prečo

Lepšia organizácia modelu, jednoduchšia orientácia v systéme, riadenie závislostí medzi časťami

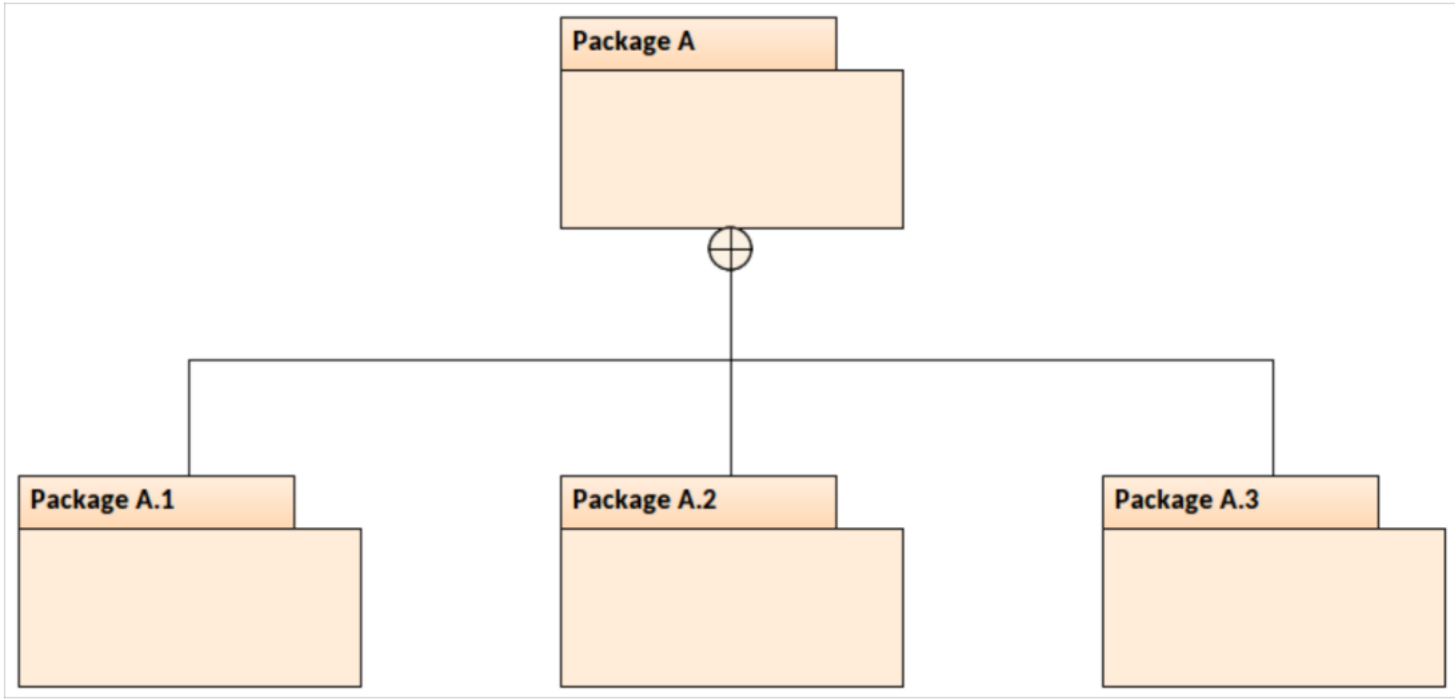
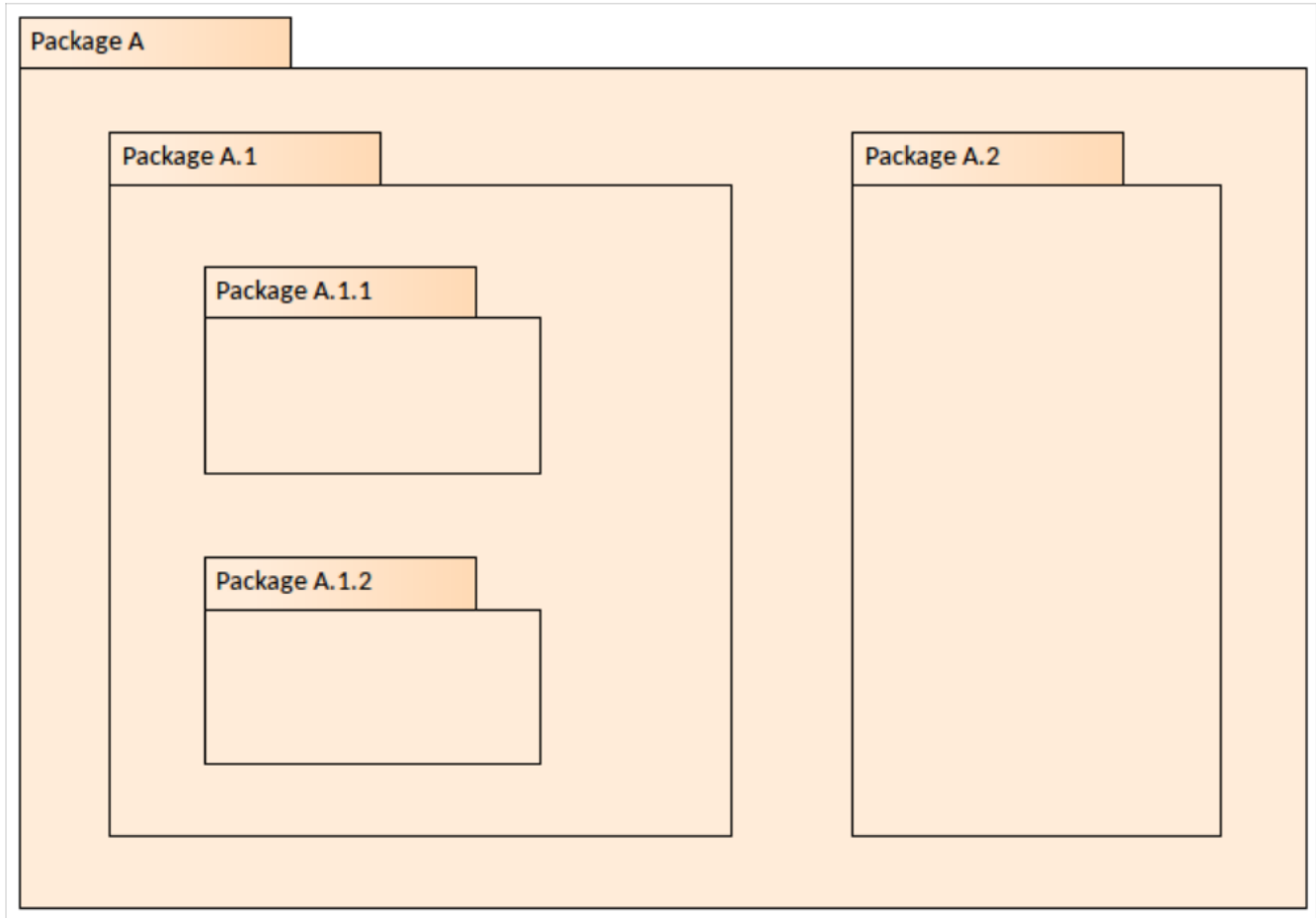


Diagram kompozitnej štruktúry (Composite Structure Diagram)

Zobrazuje **vnútornú štruktúru triedy, komponentu alebo iného prvku** a z akých častí sa skladá a ako spolu komunikujú.
Ide hlbšie ako diagram tried.

V diagrame zobrazujeme **interné časti, porty, konektory** a **vzťahy** medzi **vnútornými časťami** modelovaného prvku.

Pre koho a prečo

Softvéroví architekti, analytici a vývojári, ktorí potrebujú detailne pochopiť vnútorné usporiadanie zložitejšieho prvku a presnejšie navrhnuť komunikáciu medzi jeho časťami.

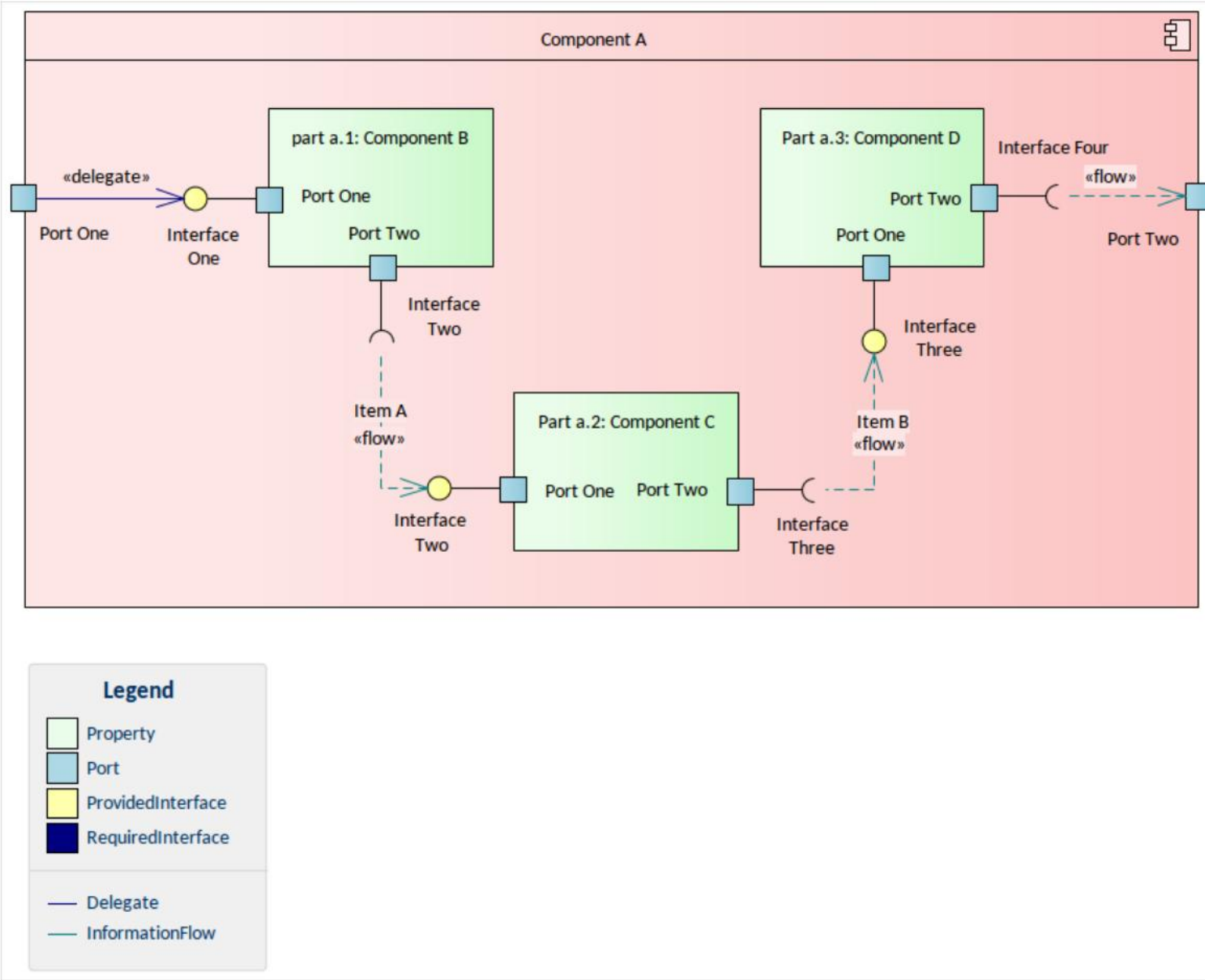
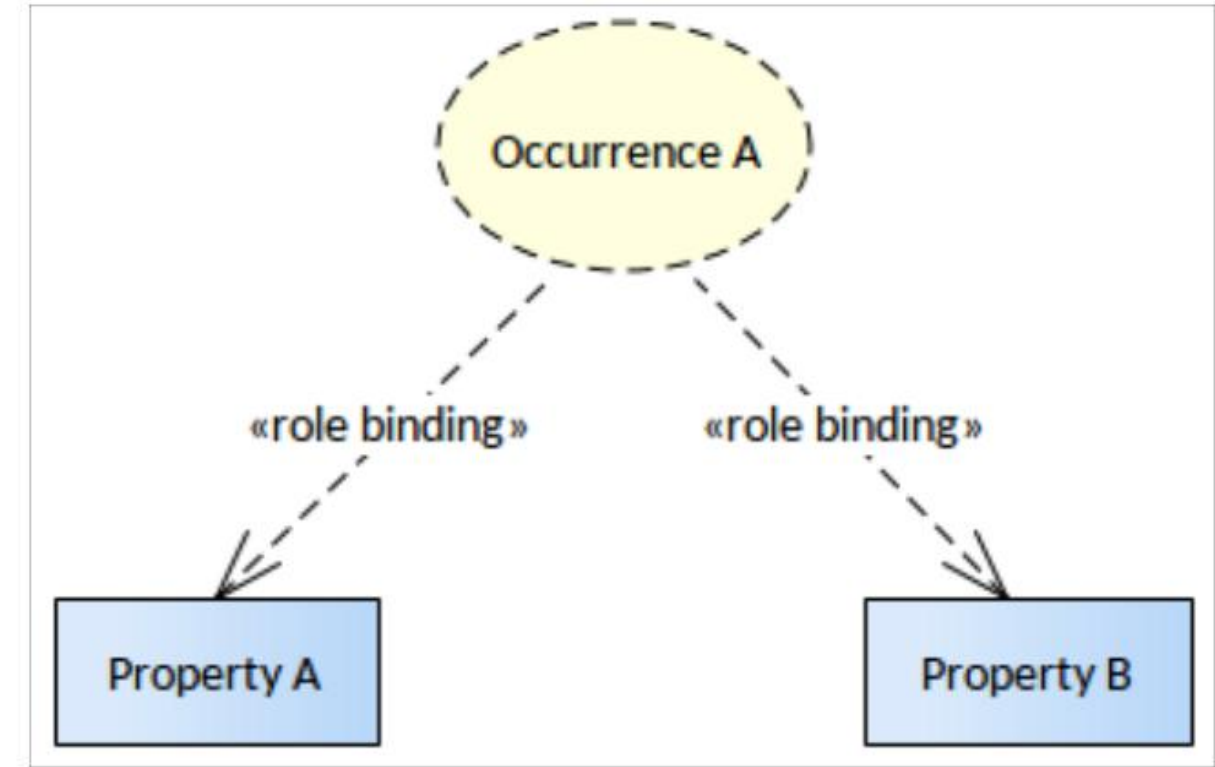
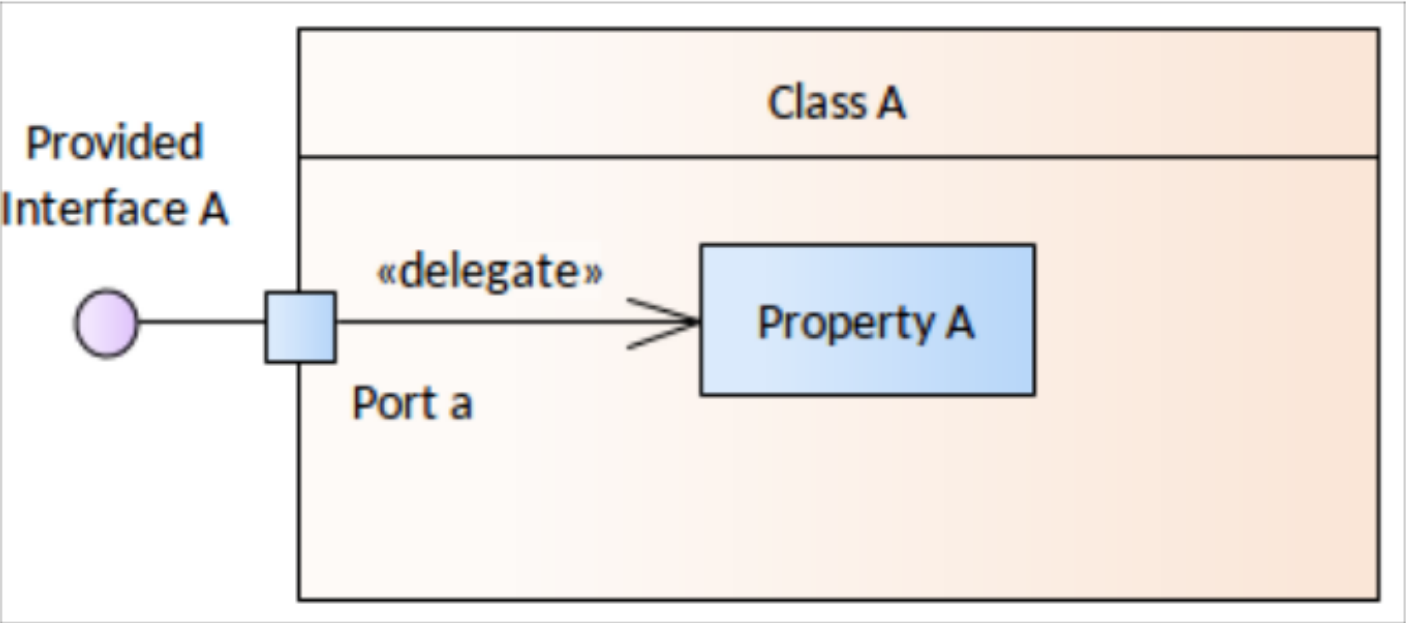


Diagram profilov (Profile Diagram)

Umožňuje **rozšíriť štandardný UML jazyk** pre **konkrétnu doménu, technológiu** alebo **metodiku** pomocou **stereotypov, značiek** a **obmedzení**.

Pre koho

Metodici,
softvéroví
architekti,
analytici a tímy
prispôsobujúce
UML
špecifickému
prostrediu

Kde sa používa

Podniková
architektúra,
bezpečnosť,
databázy,
vstavané
systémy,
špecializované
technológie

Prečo

Prispôsobenie
UML
konkrétnemu
projektu pri
zachovaní
štandardizovan
ého spôsobu
modelovania

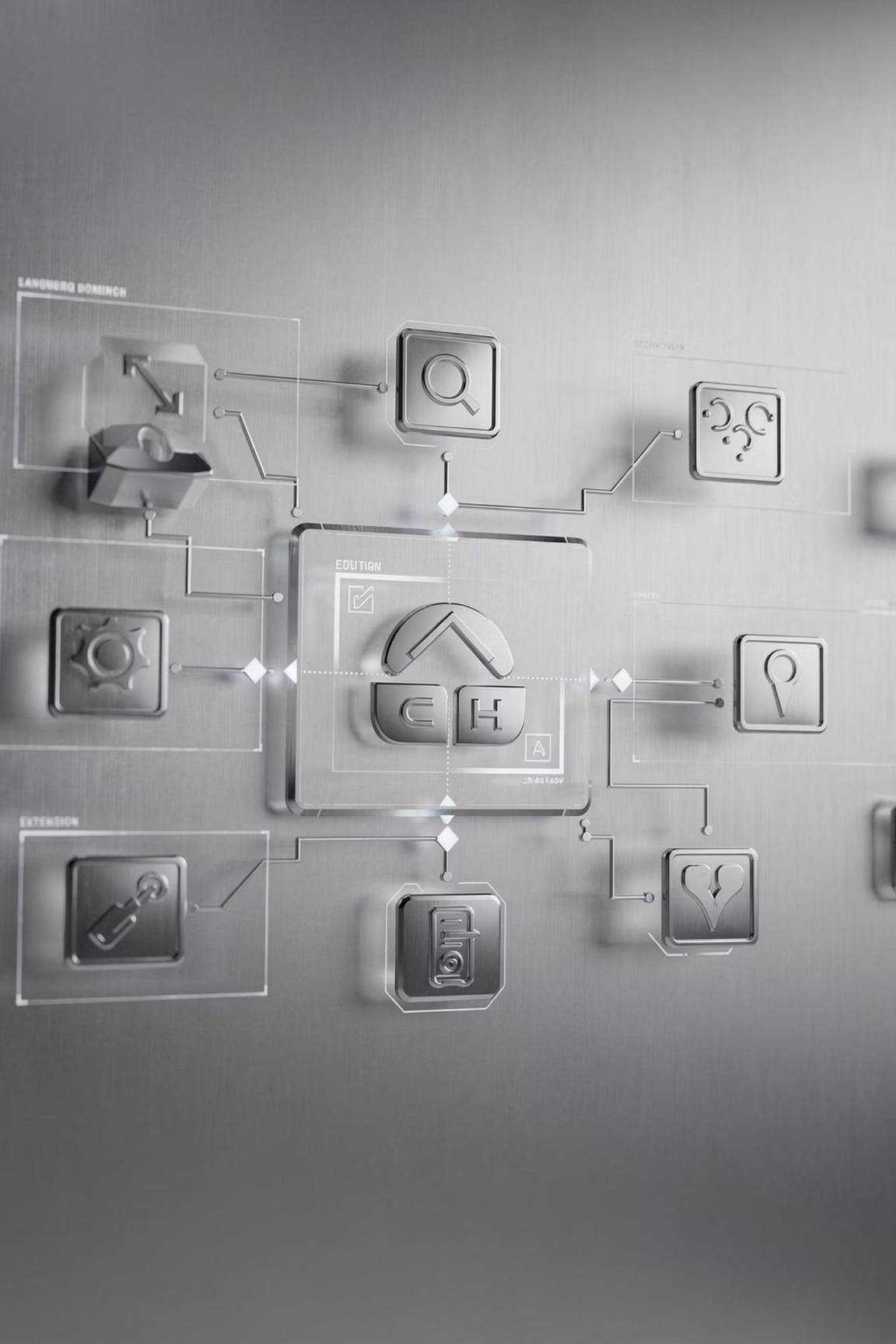


Diagram prípadov použitia (Use Case Diagram)

Zobrazuje, **čo má systém poskytovať používateľom** alebo **externým systémom**. Zameriava sa na funkcie systému z pohľadu používateľa ako sú aktéri, prípady použitia a vzťahy medzi nimi.

Pre koho

Zákazníci,
analytici,
projektoví
manažéri,
vývojári,
testerí

Kde a kedy

Analýza
požiadaviek,
komunikácia so
zákazníkom;
najmä na
začiatku
projektu

Prečo

Jasné zachytenie
funkčných
požiadaviek a
zjednodušenie
komunikácie
medzi tímom a
zadávatelom

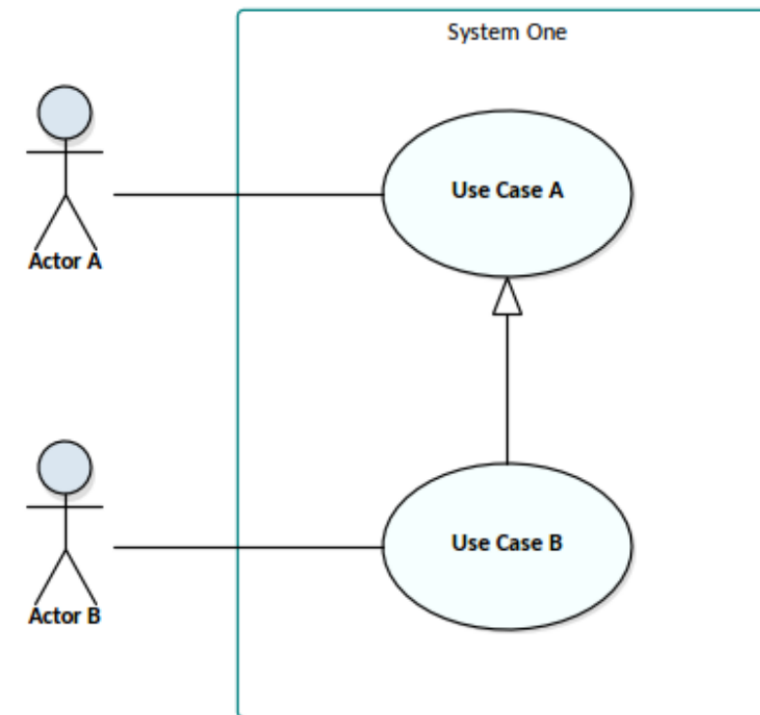
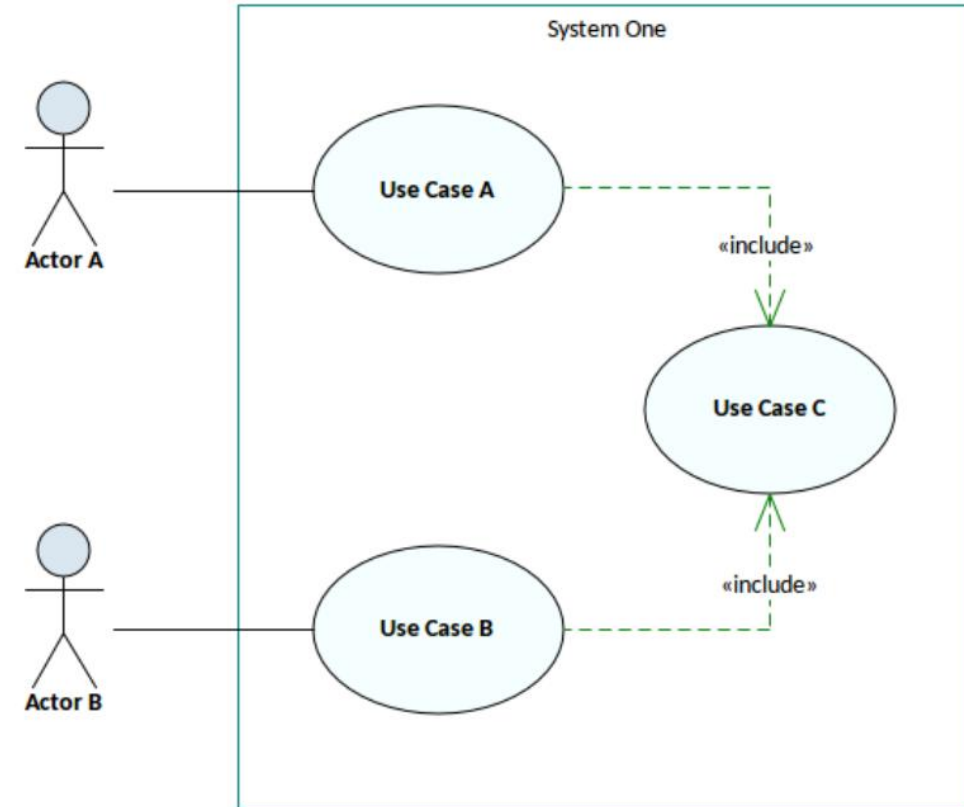
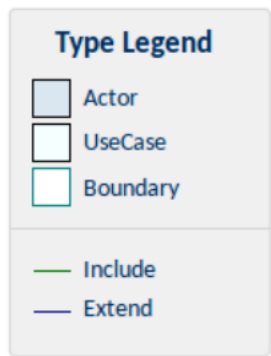
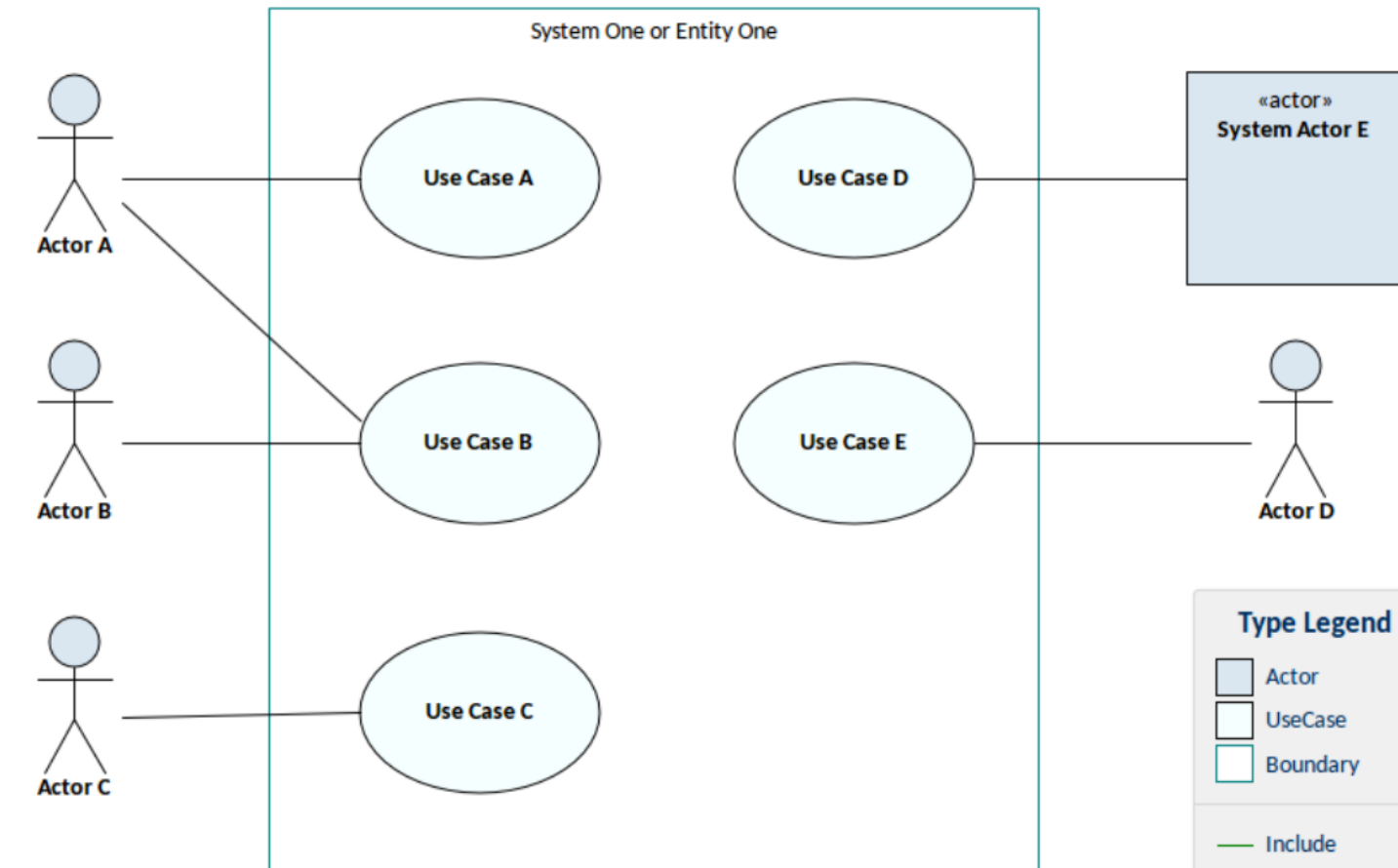
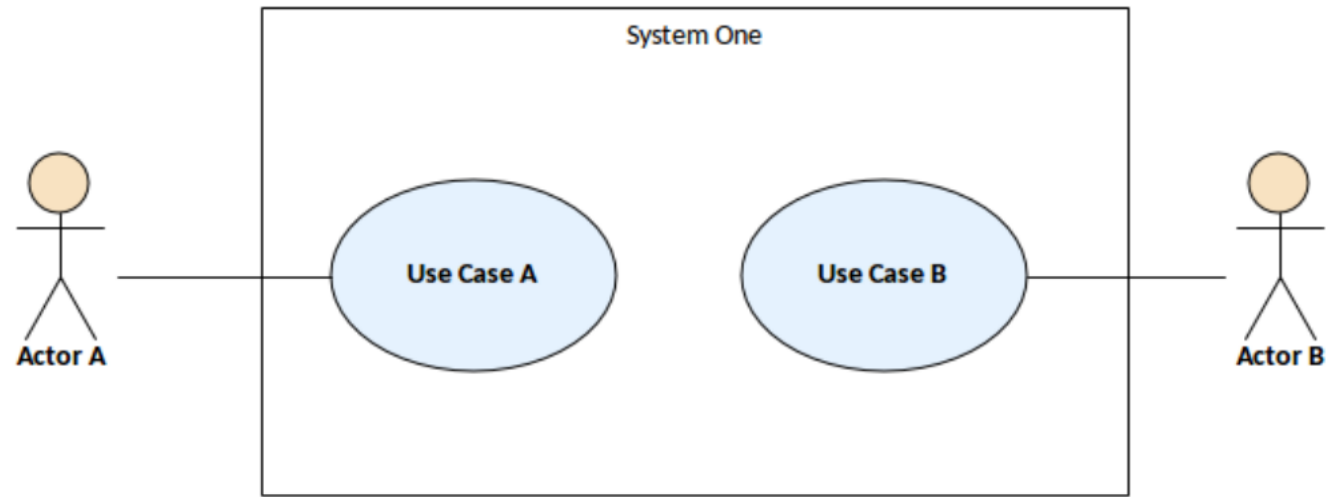


Diagram aktivít (Activity Diagram)

- Zobrazuje **postupnosť činností, rozhodnutí, vetvení a tokov** v procese alebo algoritme.
- Obsahuje **aktivitu, prechody, rozhodovacie uzly, začiatok, koniec** a **plavecké dráhy** pre zodpovednosti rolí.
- Používa sa pri **modelovaní biznis procesov, pracovných postupov, algoritmov** a **logiky aplikácie**.

Pre koho

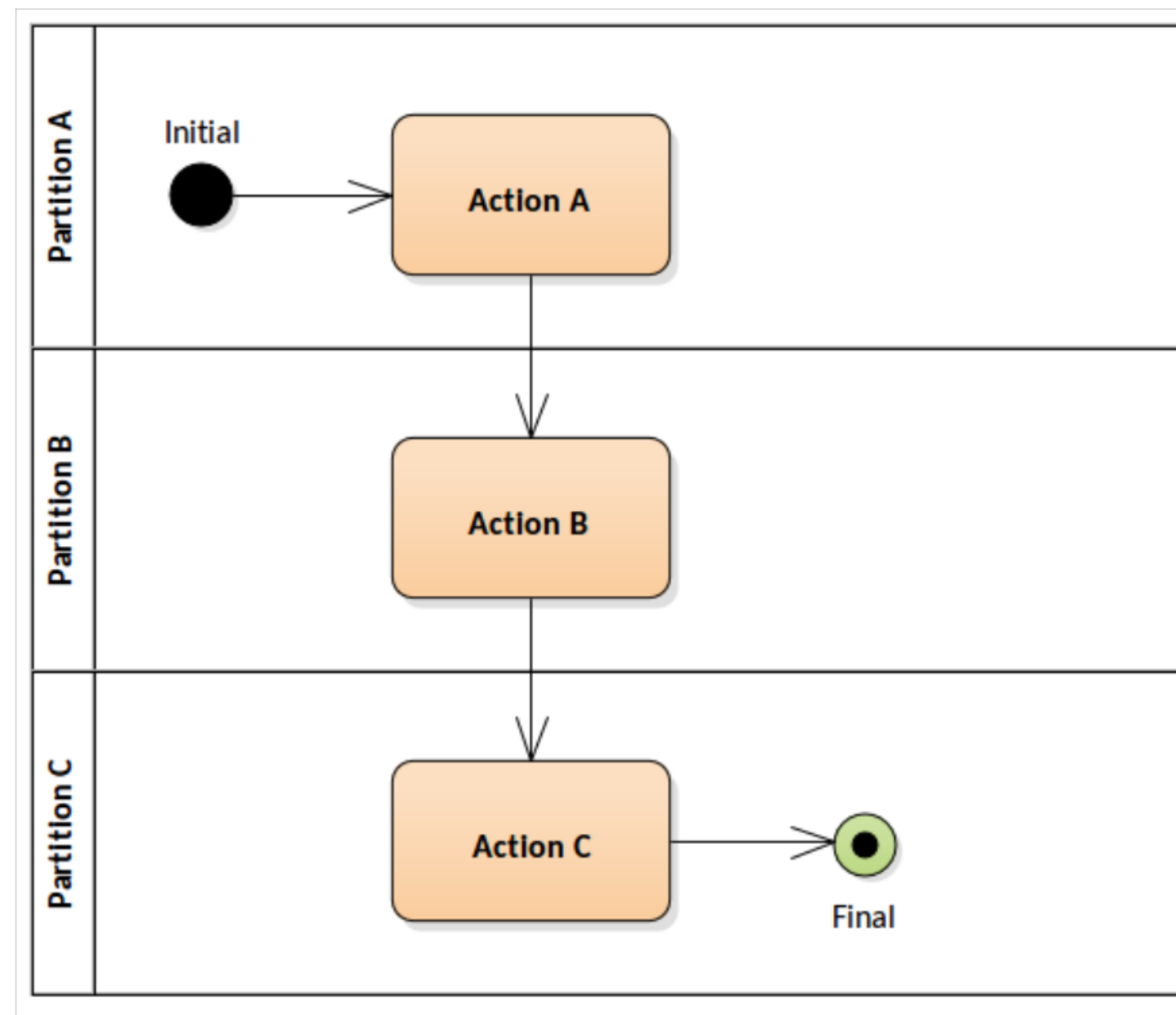
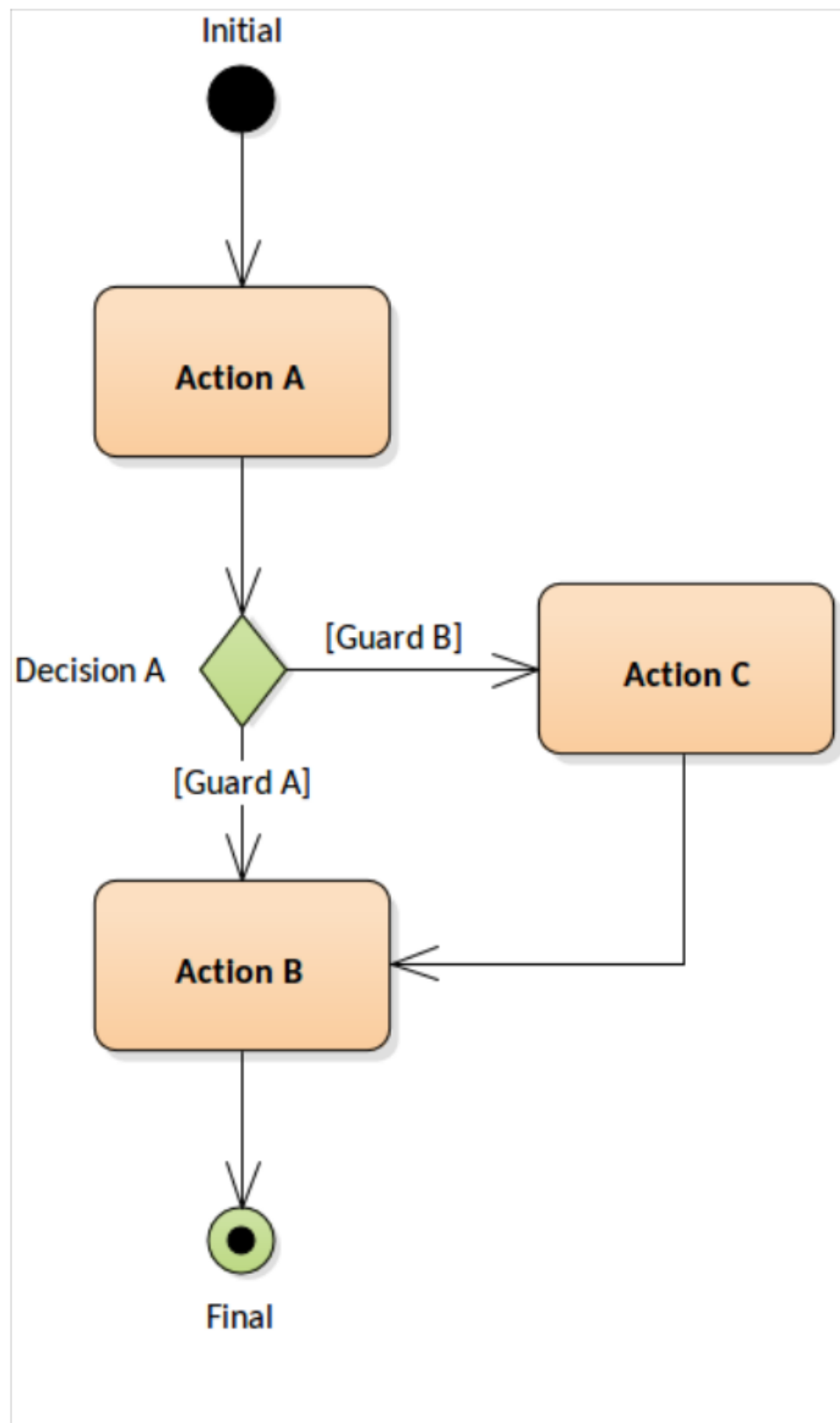
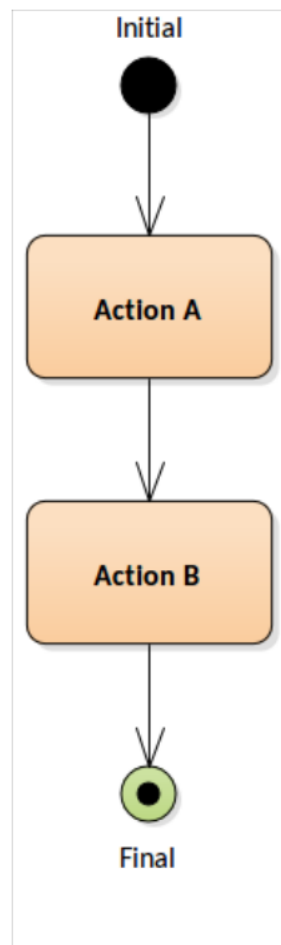
Analytici, vývojári, testerí, procesní manažéri, študenti

Kedy

Keď potrebujeme opísať priebeh procesu od začiatku po koniec vrátane alternatívnych ciest

Prečo

Znázornenie logiky procesu, odhalenie neefektívnych krokov, podklad pre implementáciu



Stavový diagram (State Machine Diagram)

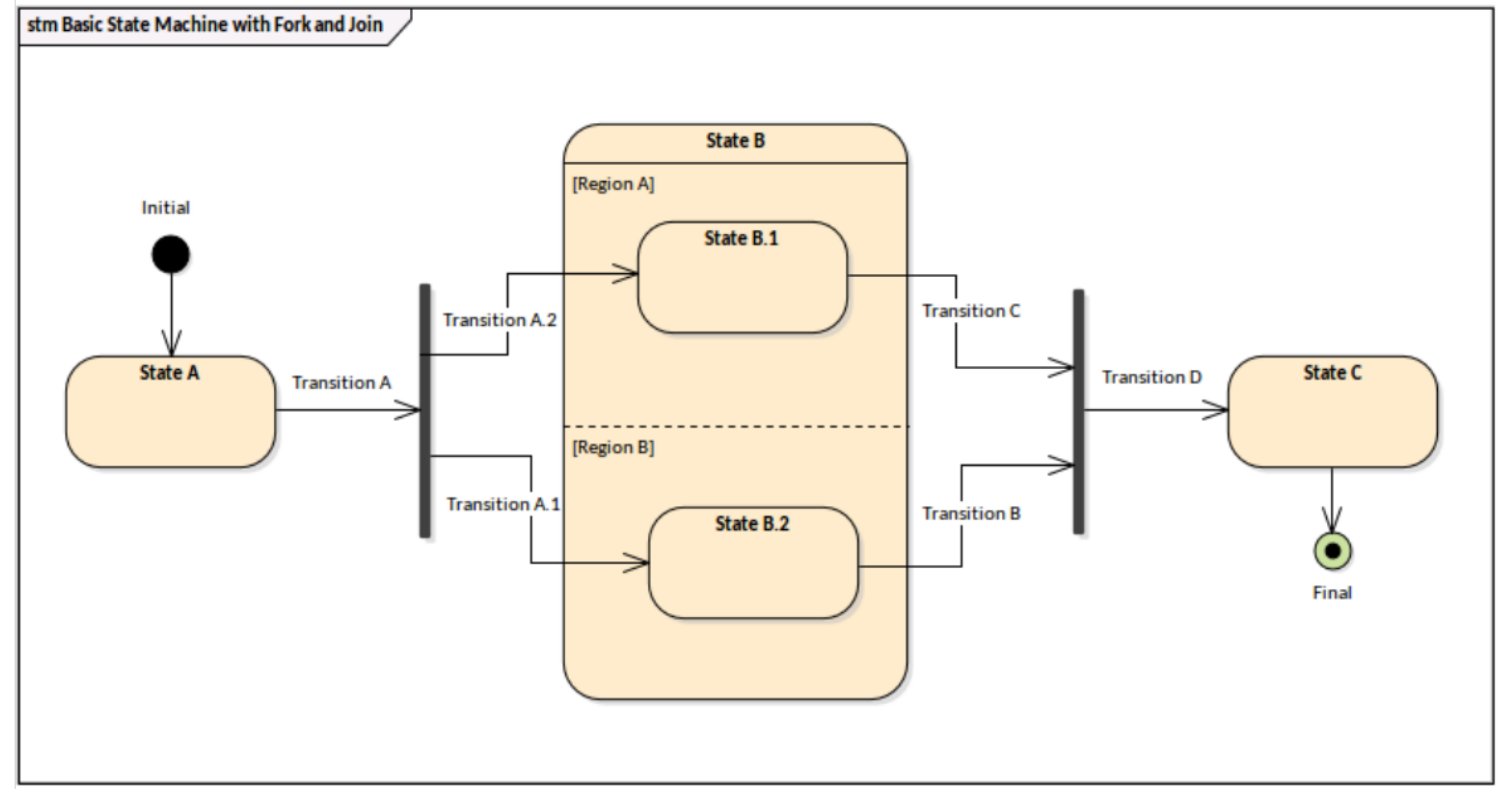
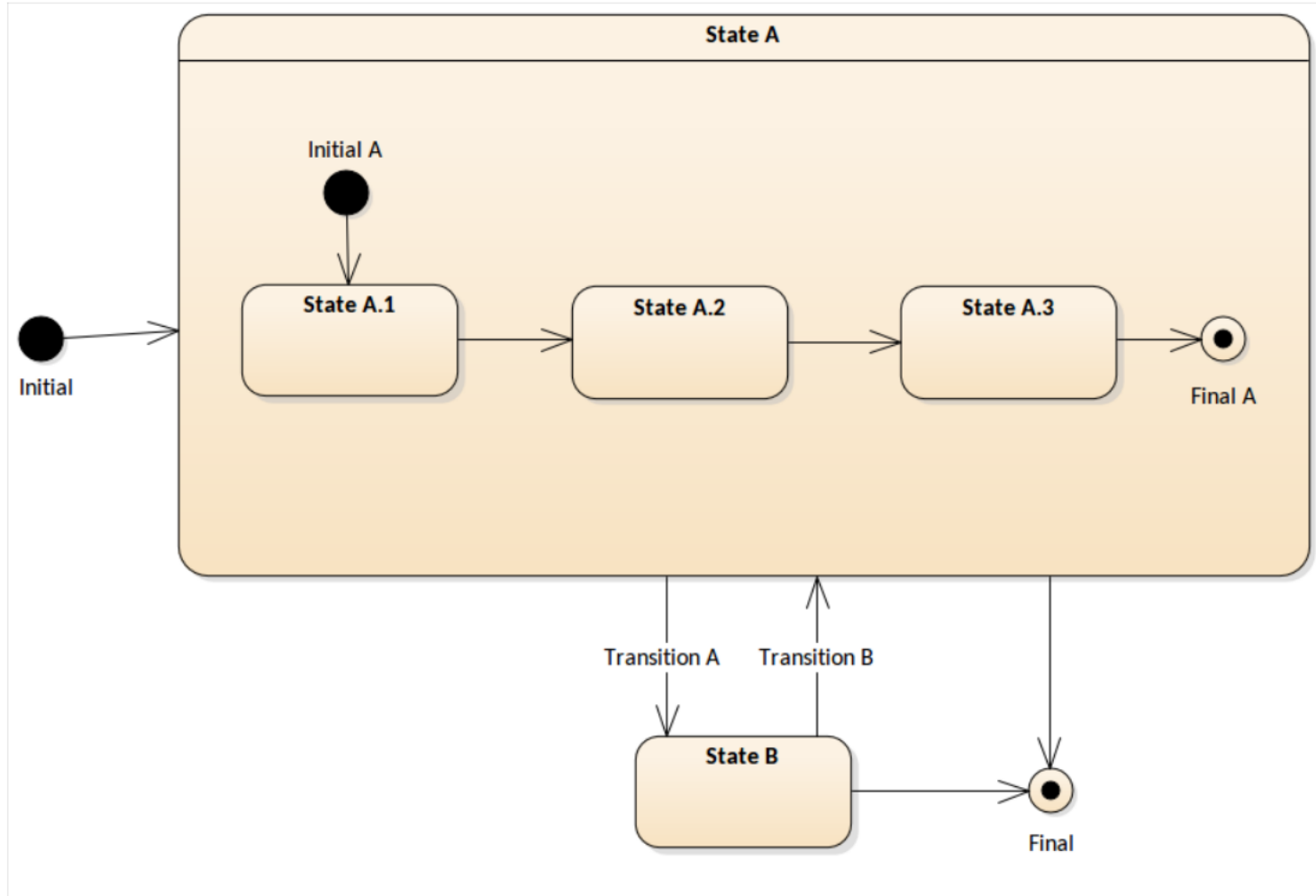
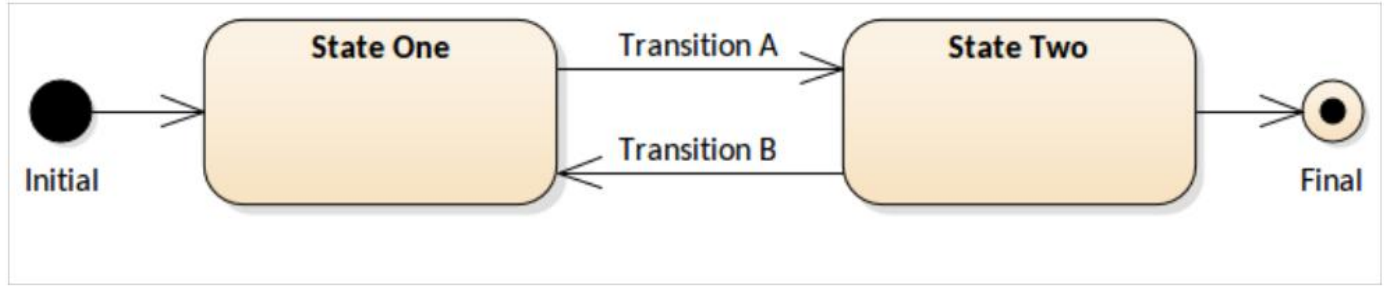
Zobrazuje **stavy objektu alebo systému a prechody** medzi nimi na základe udalostí.

Obsahuje **počiatočný stav, jednotlivé stavy, udalosti, podmienky, akcie a prechody**.

 **Kde sa používa**
Životný cyklus objektov – objednávka, platba, používateľský účet, reklamácia, zariadenie

 **Prečo sa používa**
Presný opis životného cyklu objektu, predchádzanie neplatným stavom, lepší návrh pravidiel správania





Sekvenčný diagram (Sequence Diagram)

Čo to je

- Zobrazuje komunikáciu medzi objektmi alebo komponentmi v čase s dôrazom na poradie správ.
- Obsahuje účastníkov, životné línie, správy, odpovede a aktivácie.

1 Pre koho

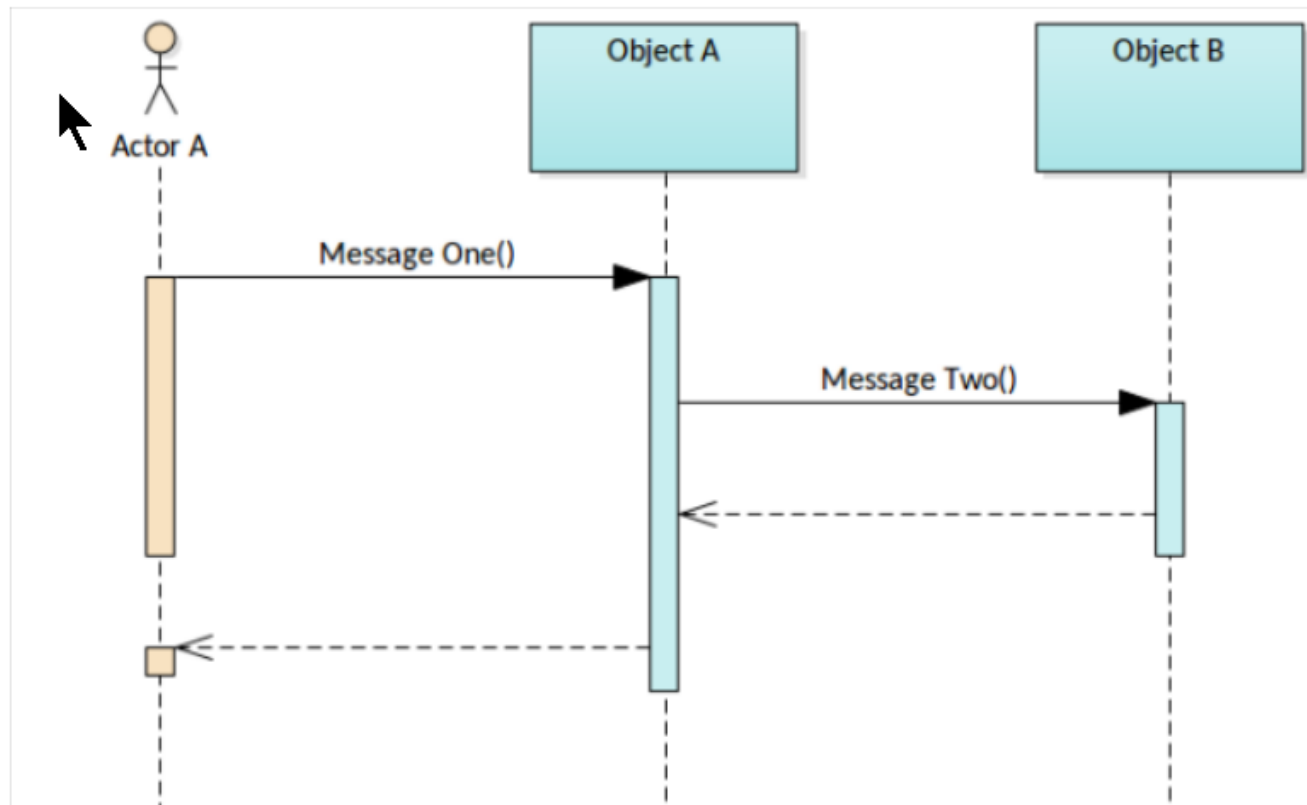
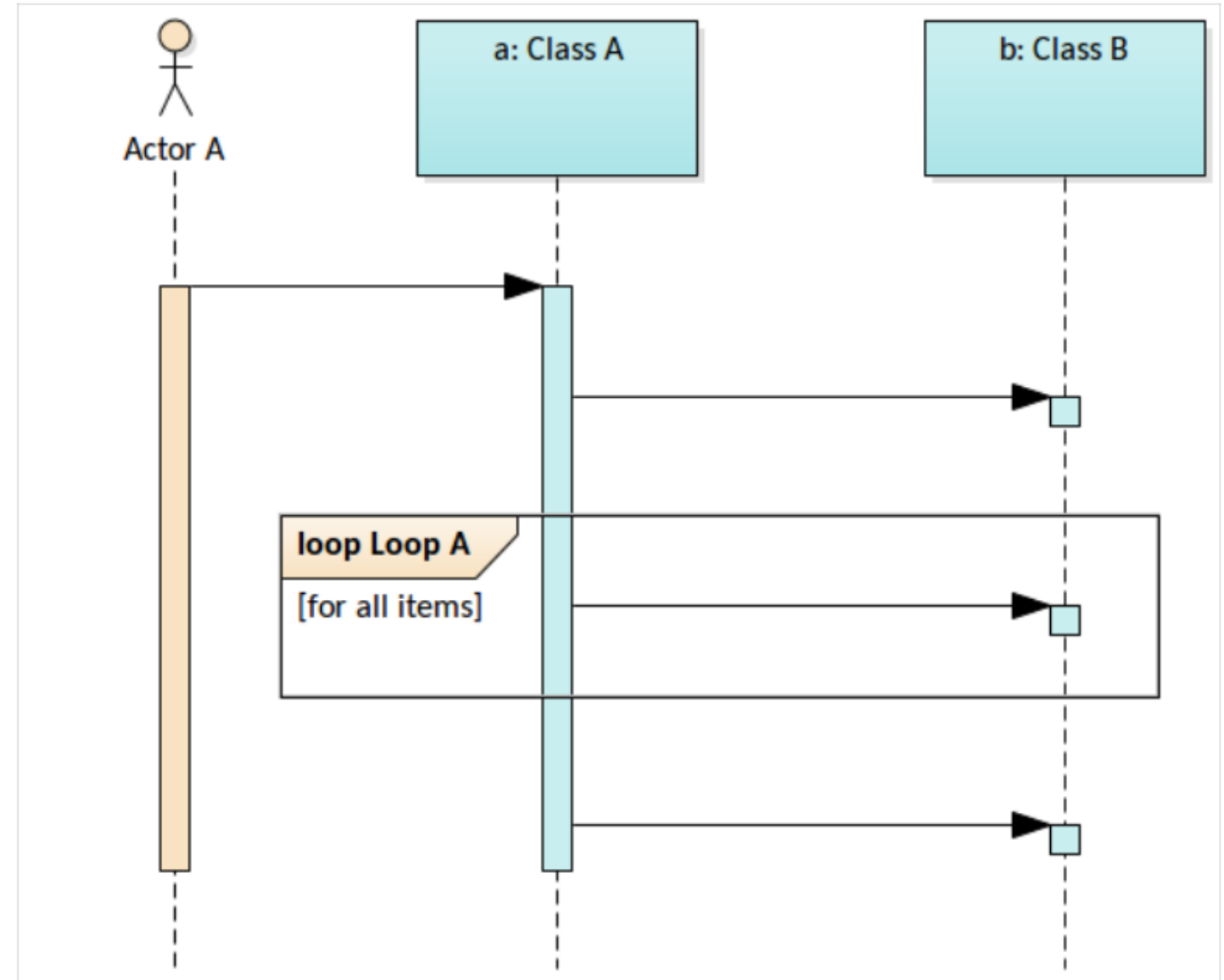
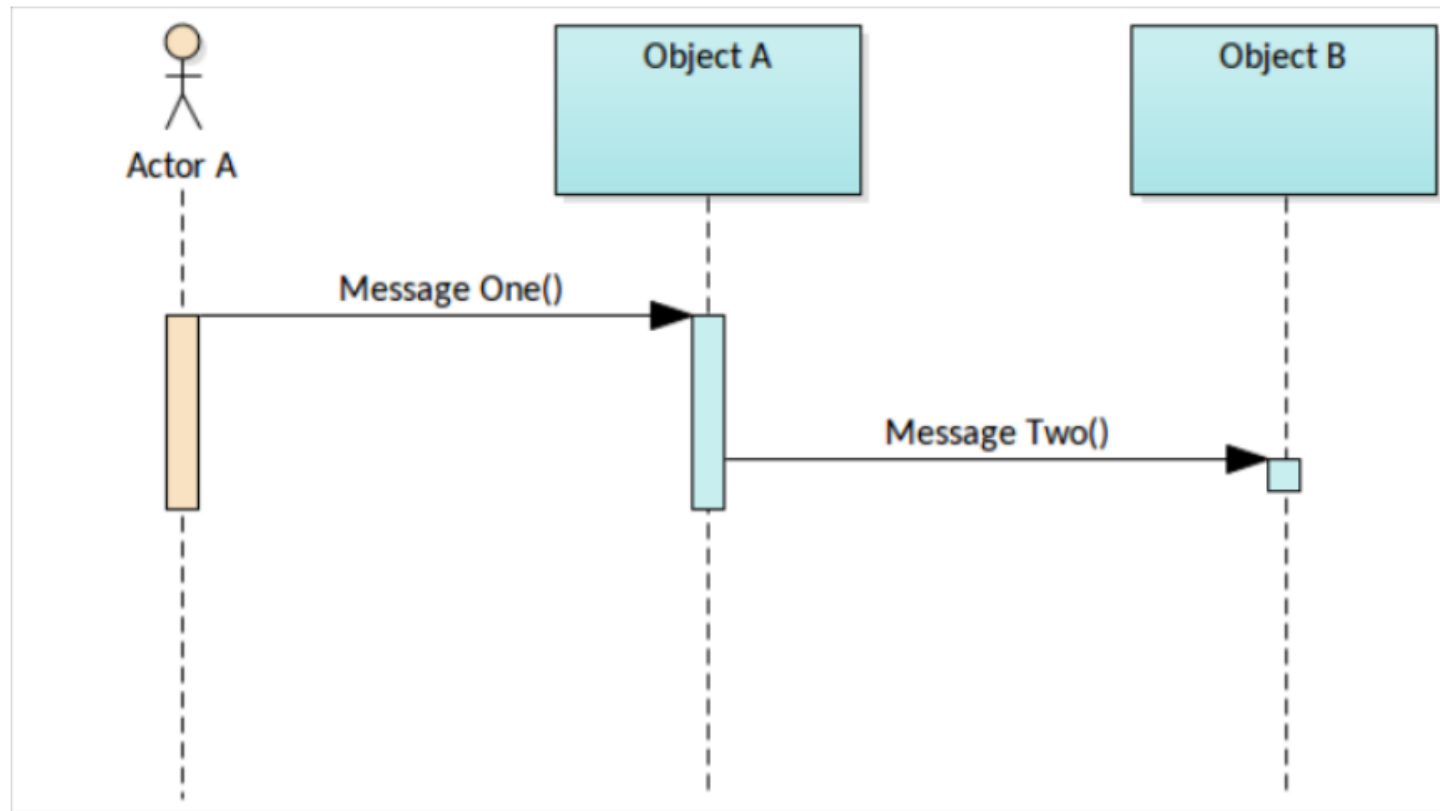
Analytici, vývojári, architekti, tester, technickí konzultanti

2 Kde sa používa

Interakcie medzi objektmi, službami, aplikačnými vrstvami alebo mikroslužbami

3 Prečo

Presný návrh toku komunikácie, identifikácia zodpovedností, podklad pre implementáciu



Komunikačný diagram (Communication Diagram)

Zobrazuje komunikáciu medzi objektmi s **dôrazom na ich vzájomné prepojenia a štruktúru spolupráce**. Na rozdiel od sekvenčného diagramu menej zdôrazňuje čas.

Pre koho
Analytici,
vývojári,
architekti,
študenti
objektového
návrhu

Ako sa používa
Objekty, väzby
medzi nimi a
očíslované
správy určujúce
poradie
komunikácie

Prečo
Lepšie
pochopenie
spolupráce
objektov a ich
zodpovedností v
konkrétnom
scenári



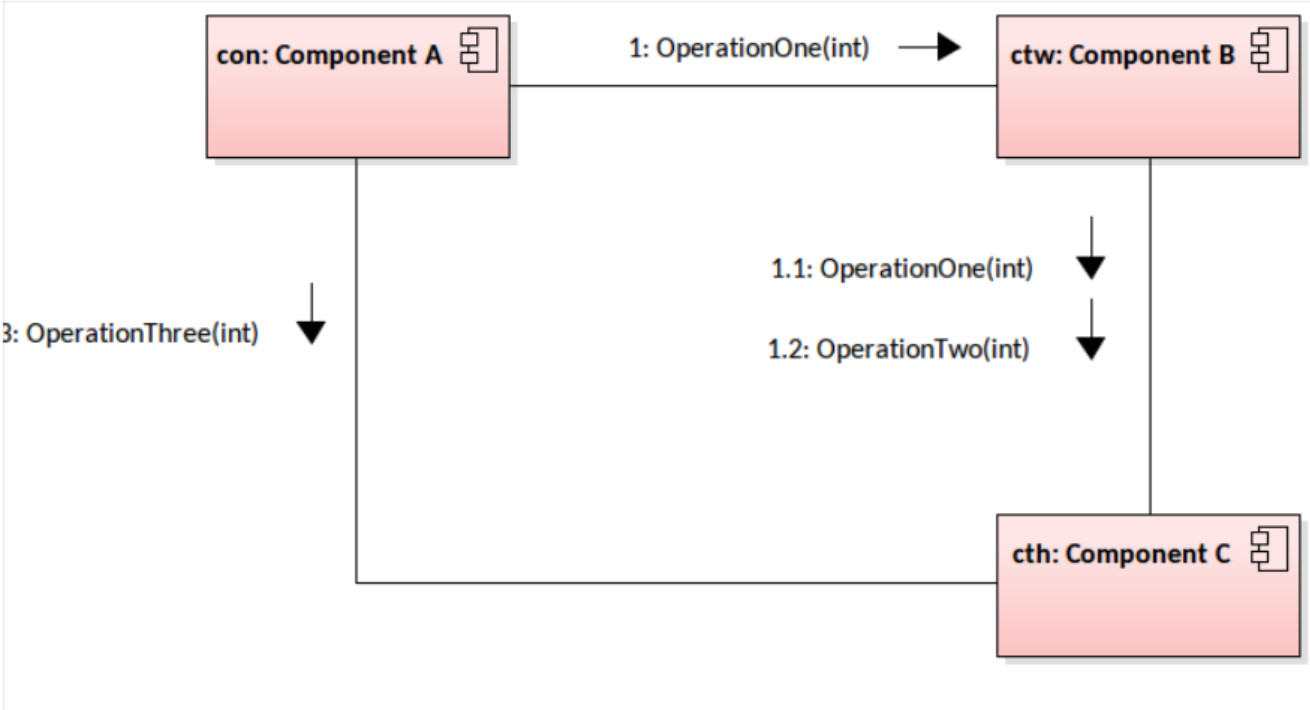
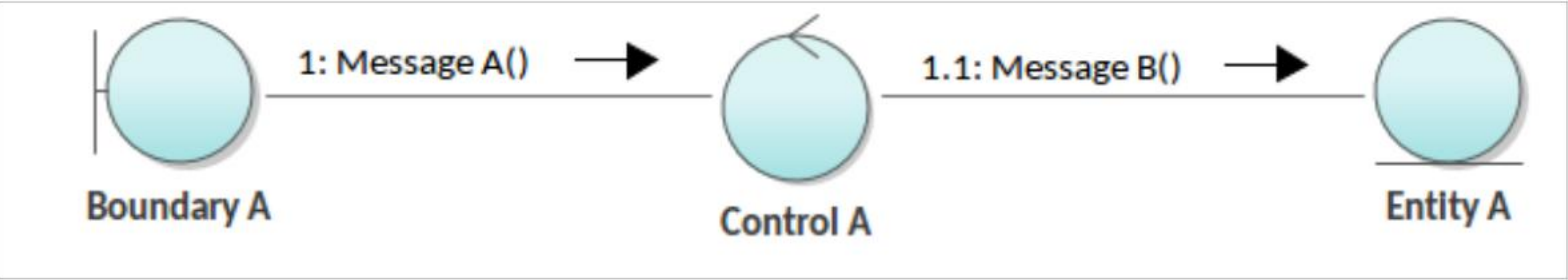


Diagram prehľadu interakcií

(Interaction Overview Diagram)

- Kombinuje prvky **diagramu aktivít a interakčných diagramov**.
- Poskytuje prehľad o tom, **ako na seba nadväzujú jednotlivé interakcie v širšom procese**.
- Tok je **podobný diagramu aktivít**, ale **jednotlivé uzly môžu predstavovať celé interakcie** alebo **odkazy na iné diagramy**.



Pre koho

Analytici,
architekti,
vývojári,
projektové tímy



Kedy

Keď je jeden sekvenčný diagram príliš detailný alebo pri viacerých interakciách v širšom procese



Prečo

Prehľad nad zložitejším správaním systému bez zahltenia detailmi správ

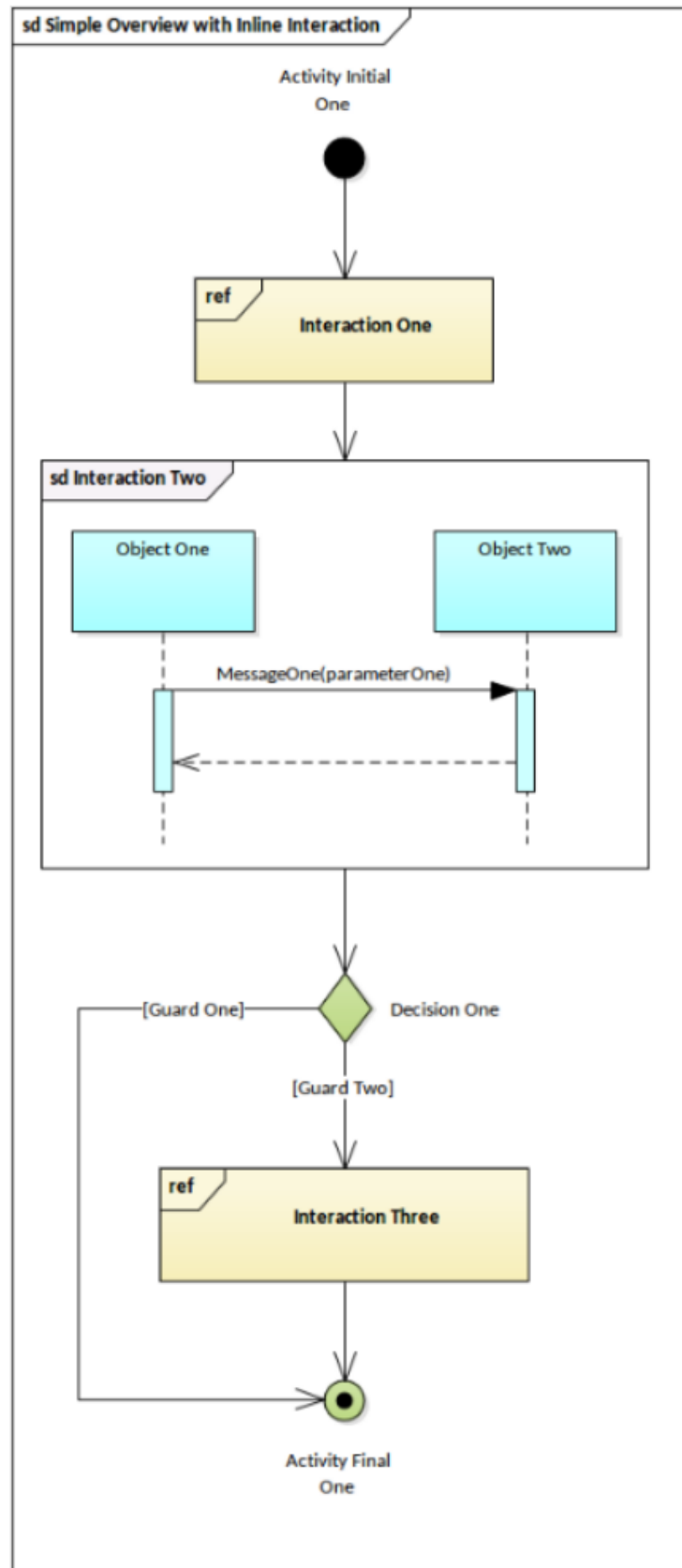


Diagram časovania (Timing Diagram)

Zobrazuje **zmeny stavu alebo hodnoty objektu v čase** s dôrazom na **časové obmedzenia** a **presný priebeh udalostí**. Obsahuje časovú os, stavy objektov, udalosti, prechody a časové intervaly.

Pre koho

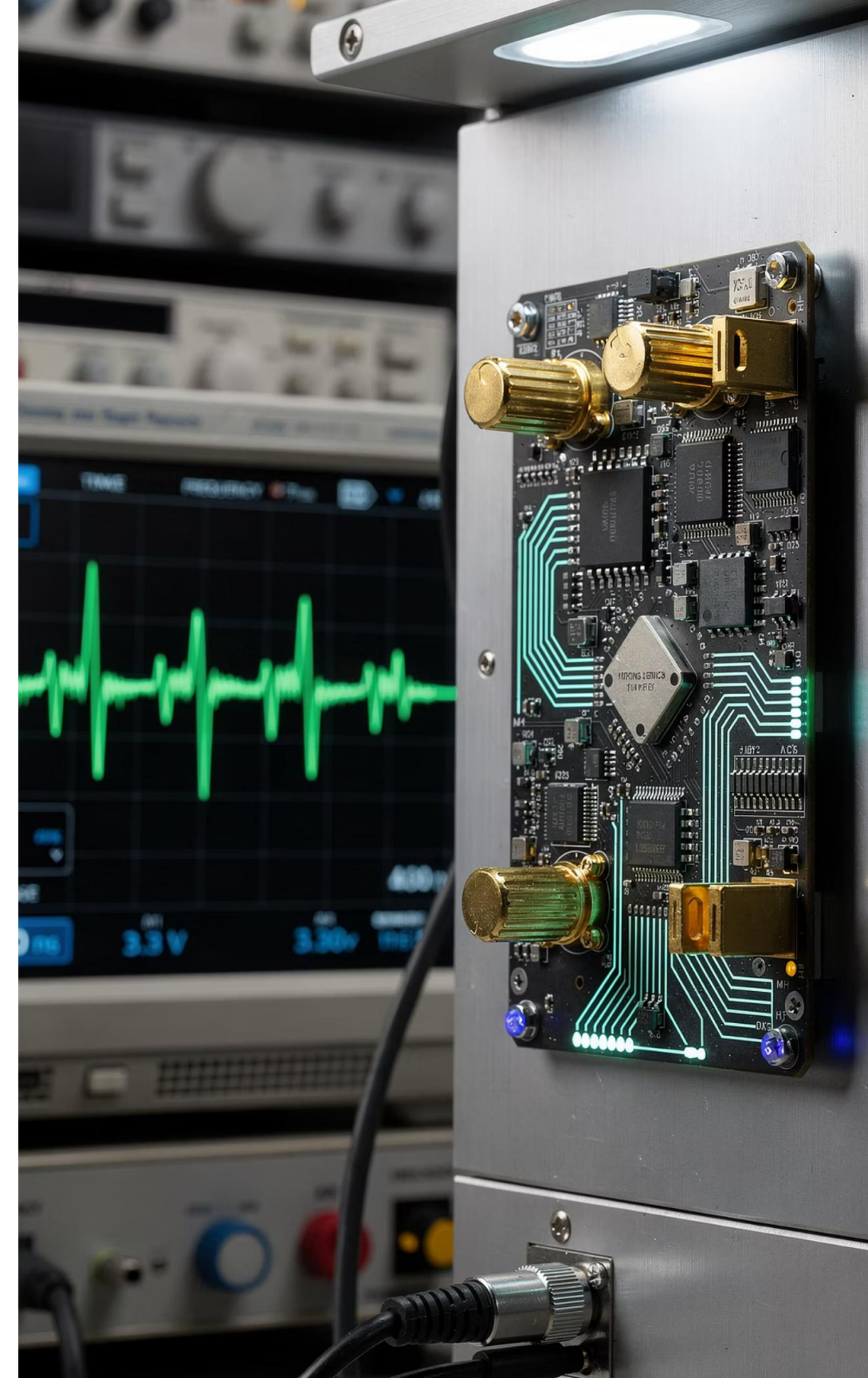
Vývojári,
architekti,
testerí,
systémoví
inžinieri, tímy
real-time
systémov

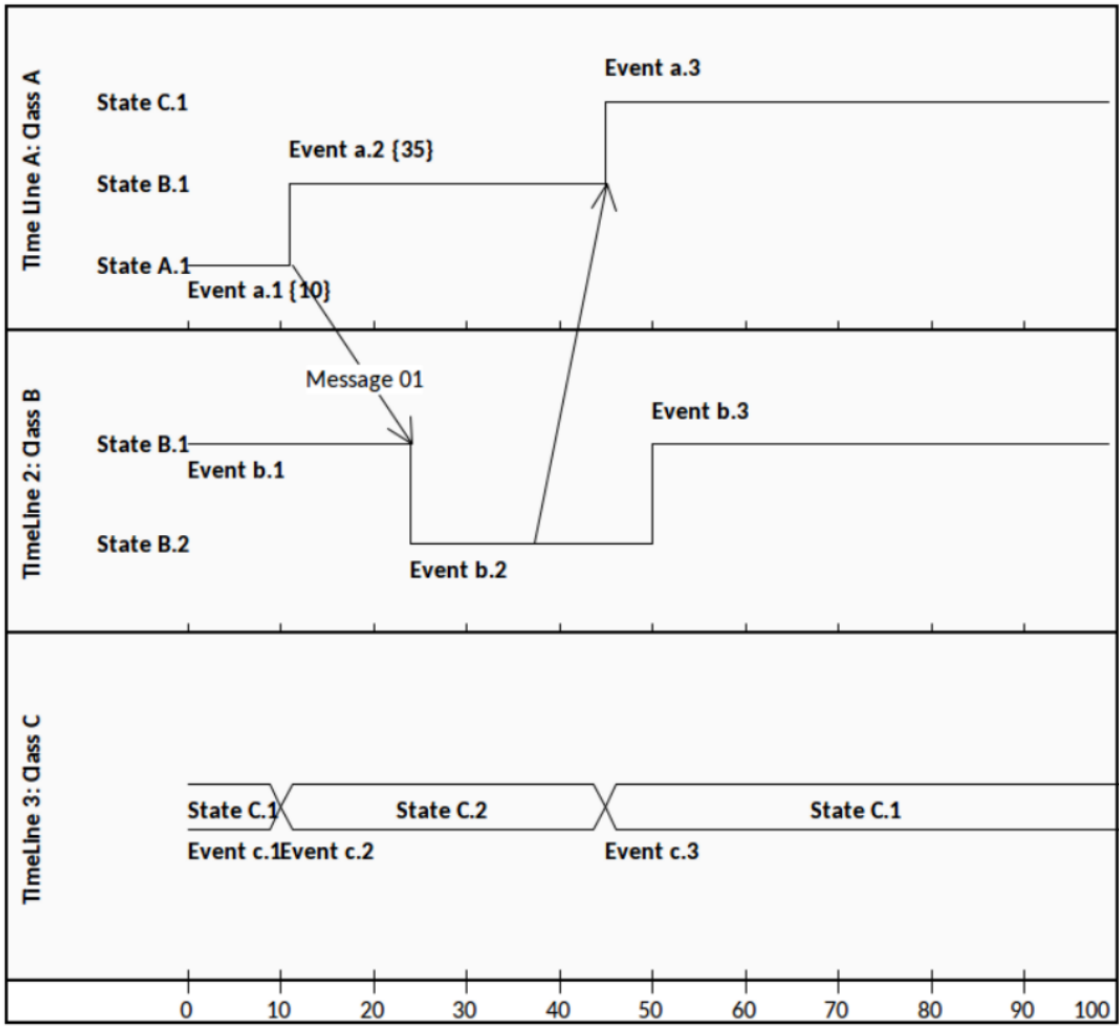
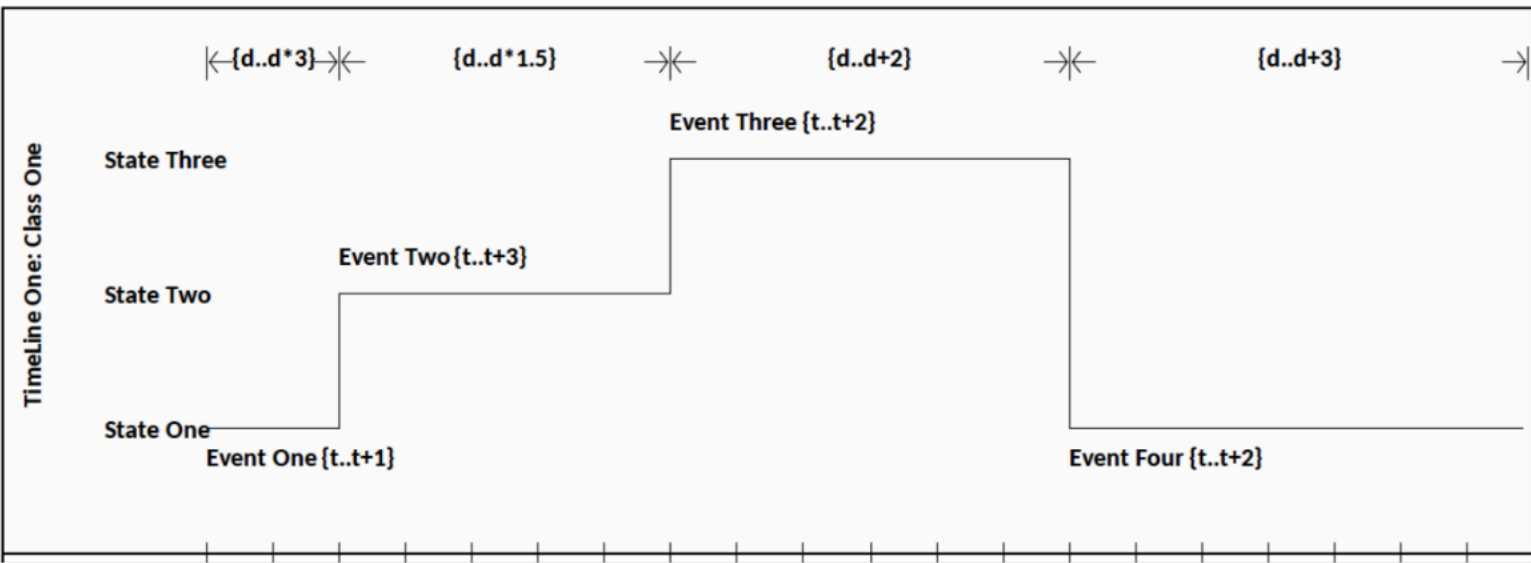
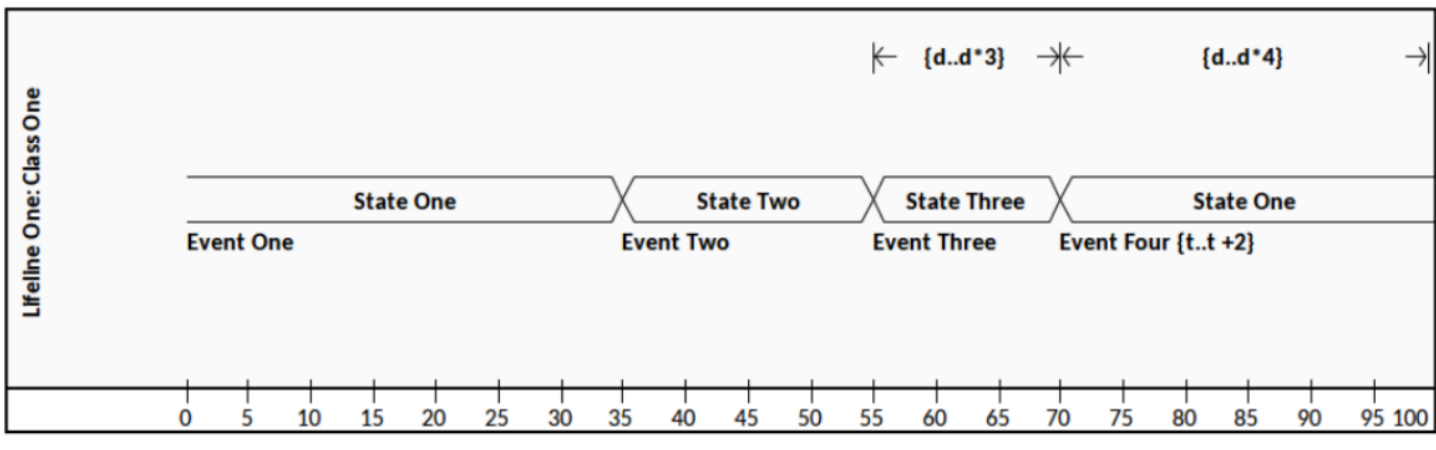
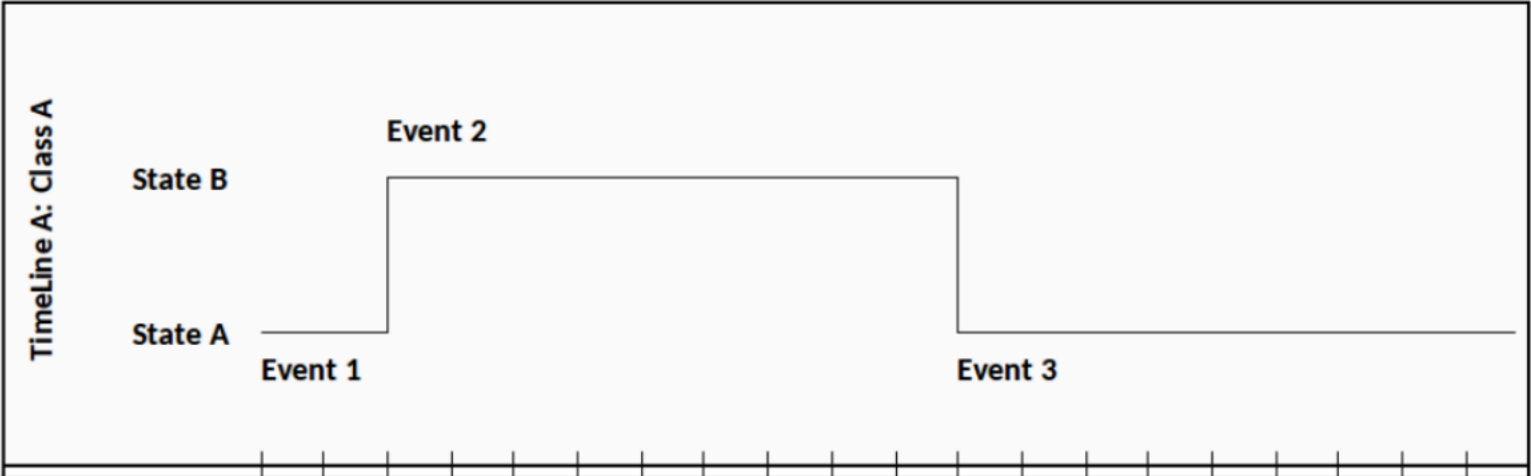
Kde sa používa

Real-time
systémy,
vstavané
systémy,
komunikačné
protokoly, HW
riadenie,
automatizácia

Prečo

Overenie časového
správania systému,
dodržanie limitov a
správna
synchronizácia
udalostí





Kedy použiť ktorý UML diagram?

Začiatok projektu

Diagram prípadov použitia
– zachytenie požiadaviek a funkcií systému

1

Architektonický návrh

Diagram komponentov,
nasadenia, profilov –
infraštruktúra a závislosti

3

Testovanie a ladenie

Diagram objektov, aktivít,
časovania – overenie
správnosti a výkonu

5

2

Analýza a návrh

Diagram tried, balíkov,
kompozitnej štruktúry –
štruktúra systému

4

Implementácia

Sekvenčný, komunikačný,
stavový diagram – detailné
interakcie a správanie

Ktorým UML Začať?

1. **Use Case Diagram** - čo má systém robiť a pre koho.
2. **Activity Diagram** - ako prebiehajú hlavné procesy.
3. **Class Diagram** - aké dátové a objektové štruktúry potrebujeme.
4. **Sequence Diagram** - ako bude prebiehať komunikácia medzi časťami systému.
5. **Component Diagram** - ako systém rozdelíme na moduly alebo služby.
6. **Deployment Diagram** - kde bude systém nasadený.

Zhrnutie UML

UML ponúka **14 typov diagramov** rozdelených do **2 kategórií** a to **štruktúrnych** a **behaviorálnych**. Každý diagram slúži špecifickému účelu a cieľovej skupine.

Štruktúrne diagramy

Opisujú statickú architektúru – triedy, objekty, komponenty, nasadenie, balíky, kompozitnú štruktúru a profily

Behaviorálne diagramy

Zachytávajú dynamické správanie – prípady použitia, aktivity, stavy, sekvencie, komunikáciu, interakcie a časovanie

- ✔ Správna voľba diagramu závisí od fázy projektu, cieľového publika a toho, čo potrebujeme komunikovať a to štruktúru alebo správanie systému.





Čo sa naučíme?

1. Čo je UML diagram?
2. Na čo sa UML diagramy používajú v praxi?
3. Ktorým UML diagramom sa v praxi najčastejšie začína?
4. Aký je rozdiel medzi štruktúrnymi a behaviorálnymi UML diagramami?
5. Na čo slúži diagram tried?
6. Aký je rozdiel medzi diagramom tried a diagramom objektov?
7. Kedy použijeme sekvenčný diagram?
8. Na čo slúži diagram aktivít?
9. Aký je rozdiel medzi diagramom komponentov a diagramom nasadenia?
10. Prečo je UML užitočné aj vtedy, keď systém už existuje?